

智元酷拓四足机器人 D1 Max

SDK 开发指南



版本: V0.1.0
时间: 2026.07

目录

一、 产品介绍	5
1.1 产品概述	5
1.2 产品清单	5
1.3 产品参数	6
1.4 背部负载安装	11
1.4.1 负载安装孔位图	11
1.4.2 重心限制	12
1.5 电气拓展接口	12
1.5.1 电源口 48V(10A)	14
1.5.2 电源口 24V(20A)	16
1.5.3 网口(24V)	17
1.5.4 网口(12V)	20
1.5.5 串口	22
1.5.6 USB 口	23
1.6 D1 Max 硬件架构图	24
二、 SDK 开发指南	25
2.1 SDK 介绍	25
2.2 SDK 软件服务接口列表	25
2.3 安全保护机制	26
2.3.1 硬急停	26
2.3.2 软急停	26
2.3.3 内部异常保护(自动触发)	26
2.4 环境依赖	27
2.5 设备登录	27
2.6 SDK 获取	29
2.7 Demo 运行	29
2.8 SDK 控制说明	30
2.9 关节命名规范	32
2.10 传感设备坐标定义	33
三、 SDK 函数介绍	34
3.1 HighLevel API 函数介绍	34
3.1.1 错误码机制	34
3.1.2 类型定义	35
3.1.3 构造与析构	36
3.1.4 连接管理接口	36
3.1.5 回调设置函数	37
3.1.6 控制接口	41
3.1.7 运动与姿态控制接口	46
3.1.8 数据配置接口	47
3.1.9 版本信息	49
四、 传感设备数据获取	51
4.1 视频流数据	51
4.2 激光雷达数据	52
4.2.1 ROS2 接口定义	52
4.2.2 获取激光雷达数据	53
4.3 超声波雷达数据	53
4.3.1 ROS2 接口定义	53
4.3.2 获取超声波雷达数据	54
4.5 RTK 数据	54

4.5.1 RTK 简介	54
4.5.2 获取六分 RTK 服务	54
4.5.3 NX 侧 RTK 服务配置	55
4.5.4 ROS2 接口定义	56
4.5.5 获取 RTK 数据	58
4.6 IMU 数据	58
4.7 相机数据	58
五、调试说明	59
5.1 控制权切换说明	59
5.2 自主回充说明	59
5.3 ROS2 多机通信配置	61
5.4 日志查看	63
5.5 设备进程工具使用	64
5.6 连接外部 wifi	65
5.7 连接 WiFi 的情况下访问有线网口	65

文档修订记录

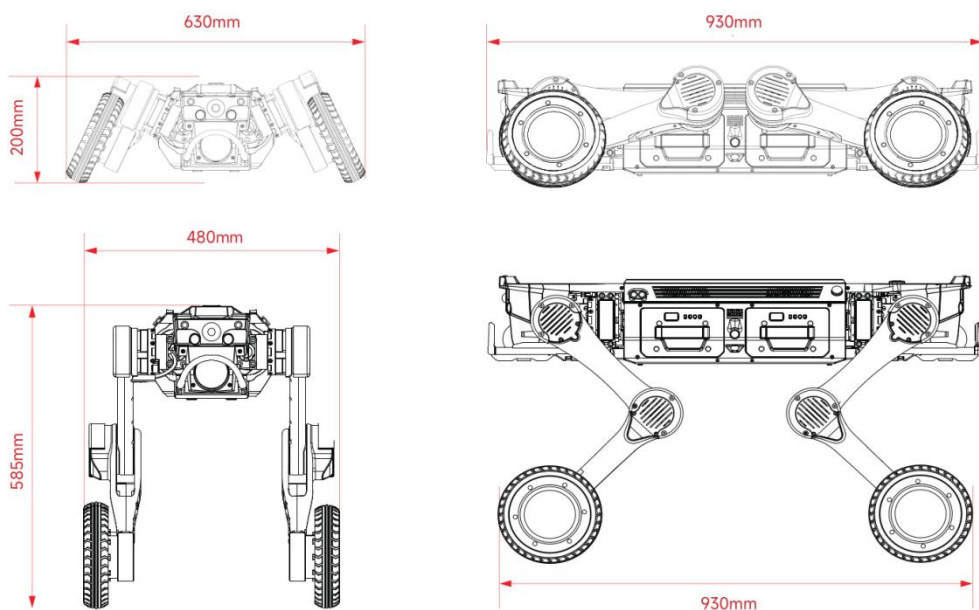
修订日期	修订内容	版本
2025.9	● 初始预测试版本	V0.0.1
2025.10	● 提供基础机器人控制接口 ● 支持机器人状态数据回调 ● 支持故障信息回调	v0.0.2
2025.11	● 添加登阶模式接口 ● 添加运动模式下爬高台接口 ● 添加模式状态获取接口	v0.0.3
2025.12.27	● 添加获取系统版本号的接口 ● 新增 UDP 通信方式支持 ● 优化连接失败时的错误信息提示 ● 内部适配新版本通信协议	v0.0.4
2026.1.17	● 新增 2.10 机体坐标系方向 ● 新增 4.4 本体传感器坐标	v0.0.41
2026.01.15	● 新增步态姿态接口(Gait(...)) ● 新增瘦身姿态接口(Slim(...)) ● 新增断开后自动续连的配置接口 ● 新增获取当前连接状态接口 ● 添加一些自动续连/手动续连的示例 ● 异步接口中 添加获取发送结果的回调参数	v0.0.5
2026.02.08	● 新增控制权切换功能 ● 新增 “4.5 RTK 使用说明” ● 修复 IMU acc/gyro 取反问题	v0.0.6
2026.03.17	● 新增自主回充功能 ● 修改 nx ssh 登录用户名 ● 修改前后激光雷达 IP 地址及 ros topic ● 新增 “1.6 D1 Max 硬件架构图”	v0.0.8

2026.03.31	● 新增单独的速度信息上报接口，并提供接口配置速度信息上报的频率 ● 适配点足的关节温度上报 ● 提高自主回充成功率 ● 增加无图回充 API 接口	v0.0.9
2026.04.30	● 新增无图自主回充 sdk 接口 ● 新增调试指南	v0.1.0
2026.07	● 勘误	v0.1.0

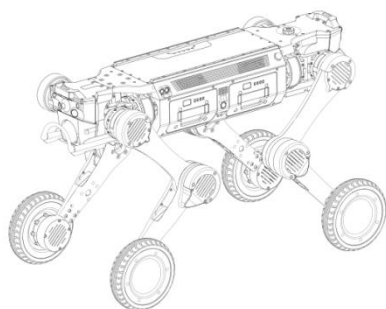
一、产品介绍

1.1 产品概述

智元仿生四足机器人 D1 Max 是一款轮足式仿生四足机器人，具备自重轻、负载大、续航长、防护强、运动灵活稳定等优势。机器人每条腿配备 3 个关节电机和 1 个轮毂电机，配备光学相机、激光雷达、超声波雷达、IMU、RTK 模块等传感设备，内部搭载高算力平台，用于实现运动控制、自主导航定位、环境侦测等功能，提供丰富的供电和通信接口，支持多品类的任务载荷拓展。



1.2 产品清单



D1 Max 本体



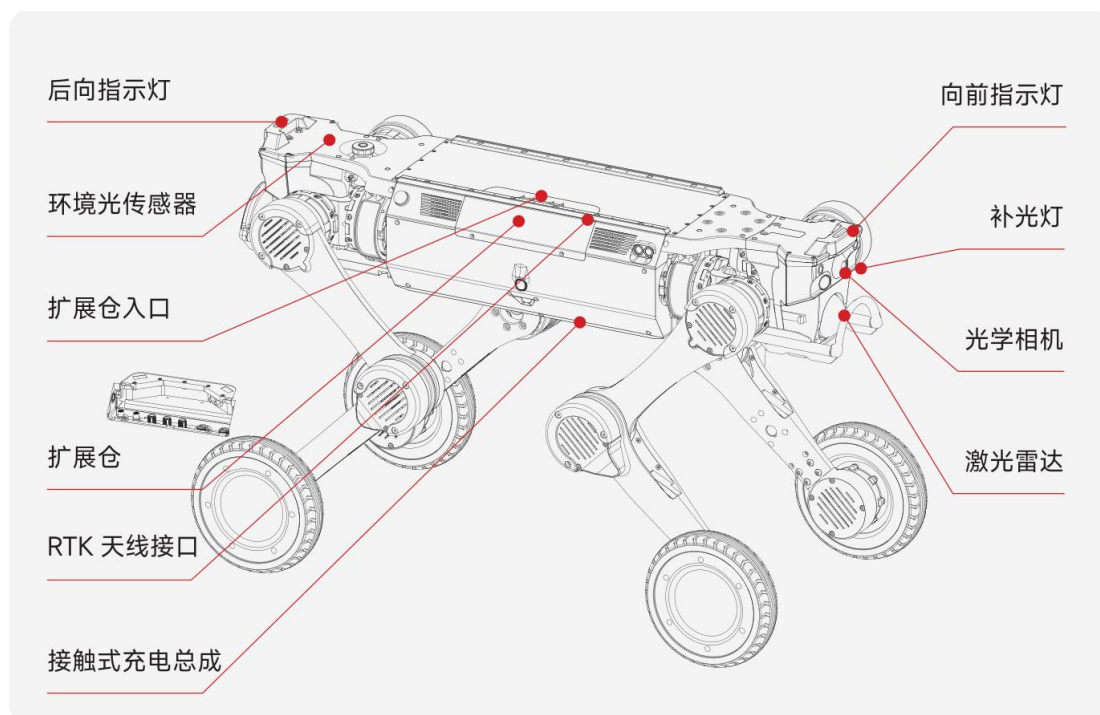
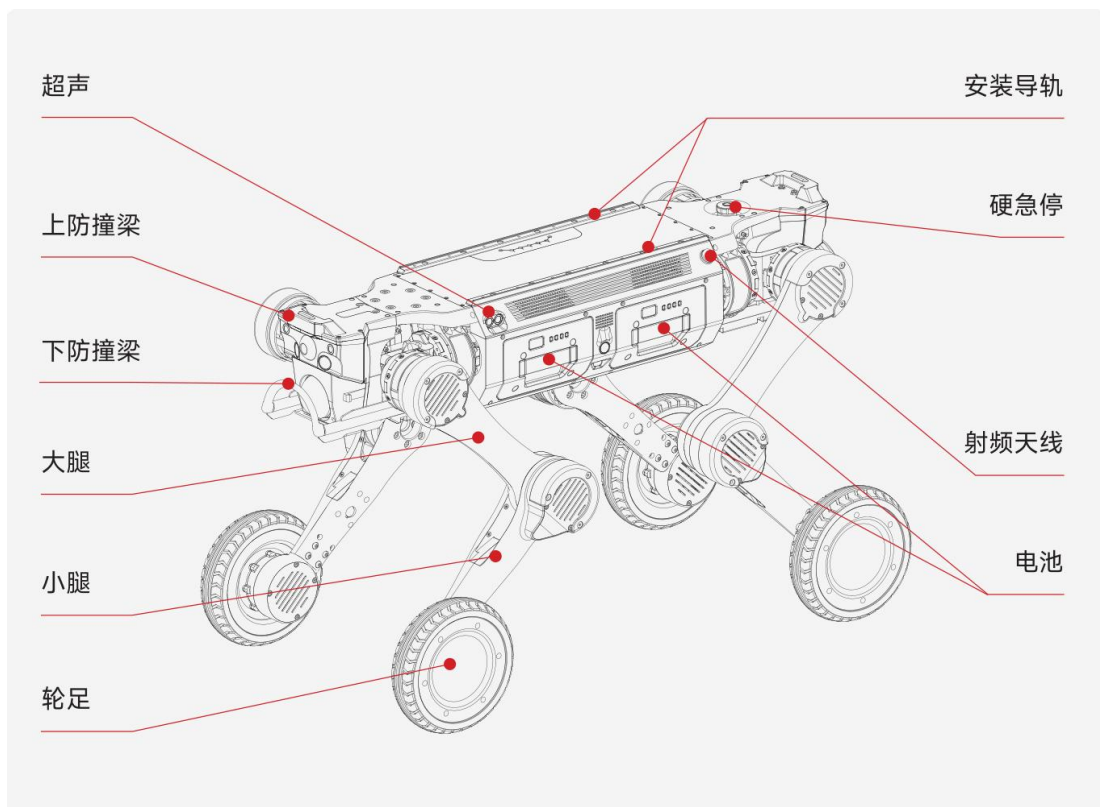
1.3 产品参数

分类	规格	说明
基础信息	站立尺寸	930mm * 480mm * 585mm
	趴下尺寸	930mm * 630mm * 200mm
	整机重量 (含电池)	41kg
	工作温度	-20°~55°
	防护等级	IP67
	电池模组	504.9Wh*2, 双电池, 支持快拆快换, 支持热插拔
	充电时长	< 1.5h
	续航时间	空载 5±0.5h, 满载 3.5±0.5h
性能参数	最大速度	6m/s
	稳定负载	30kg
	连续攀爬 台阶高度	25cm
	极限跨越 障碍高度	80cm
	极限跨越 沟壑宽度	80cm
	带负载跨 越高度	50cm
	最窄通过 宽度	50cm
	最大爬坡 角度	45°

	最窄掉头宽度	支持原地掉头，支持前后向切换
	算力主控	157 Tops
传感单元	激光雷达	前后各 1 个 96 线激光雷达(Airy96 线)、覆盖角度 360°*90°、覆盖直径范围 120m
	广角相机	前后各 1 个 FPV 相机，工业级 800 万像素，视场角：DFOV122°，HFOV111°，VFOV70°
	补光灯	补光灯*4，FPV 相机左右各 1 个。照射距离≥3m
	IMU	车规级 IMU*1
	超声波传感器	左右各一个超声波，支持 5m 以内的障碍物识别
	RTK 模块	GNSS RTK*1
	麦克风扬声器	部分版本支持
	光照传感器	部分版本支持
功能列表	运动性能	具备踏步、滑行、匍匐等姿态，能够前后左右直线移动、原地旋转等，支持膝关节姿态变换； 支持在砂石路面、柏油路面、草地、沙土地面、森林地面、建筑废墟等各类路面行走；
	轮\足快换	支持轮足、点足快换
	实时图像传输	支持
	OTA 升级	支持
	二次开发	支持，提供 SDK 开发包，机器人模型提供 3D 数模和 urdf 文件用于仿真
	供电接口	5V、12V、24V、48V，最大输出功率 480W

	通讯接口	千兆网口*2、USB3.0*2、串口*1
通信	WIFI 通信	支持
	蓝牙	支持
	射频通信	支持
配件	适配器充电座	标配
	带屏遥控器	标配，遥控距离 700-1000m
	自主充电桩	选配
	航空包装箱	标配
其他	保修期	1 年

注：以上参数为实验室测试数据，实际表现可能因使用环境、操作方式等因素有所差异，请以实际为准



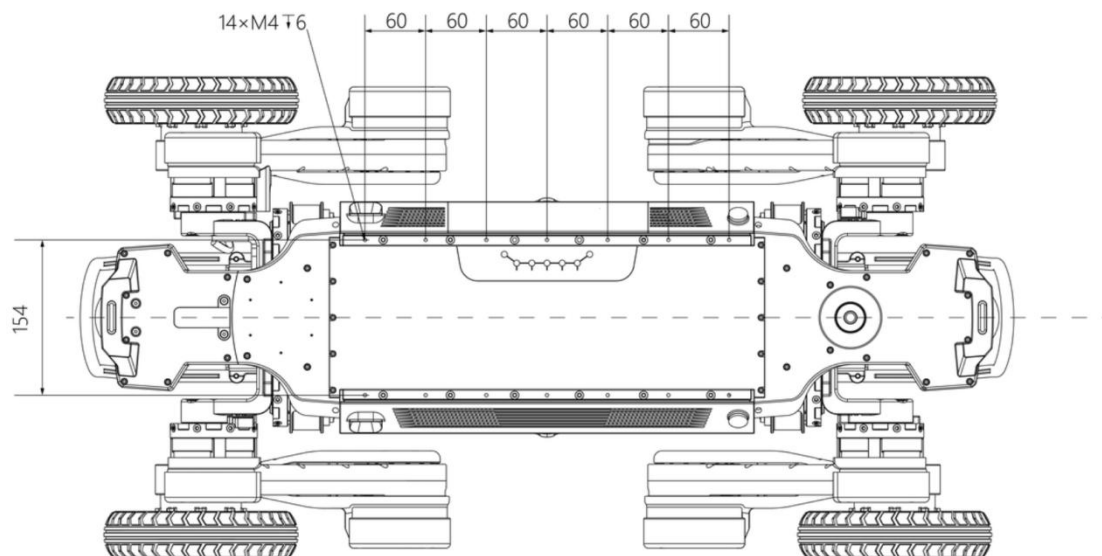
● 关节参数

部位	参数	数值
髋部滚转关节(hip_roll)	最大扭矩	150N.m
	最大转速	190RPM
	重量	0.93kg
	关节限位	-39.9°~30°
髋部俯仰关节(hip_pitch)	最大扭矩	150N.m
	最大转速	190RPM
	重量	0.93kg
	关节限位	-140°~140°
膝部俯仰(knee_pitch)	最大扭矩	150N.m
	最大转速	190RPM
	重量	0.93kg
	关节限位	-160°~160°
足端(foot)	最大扭矩	33N.m
	最大转速	1200RPM
	重量	1.275kg
	半径	9cm
	关节限位	-180°~180°

1.4 背部负载安装

1.4.1 负载安装孔位图

机器狗背部的空间平整，负载 25-30kg，背部部署有双导轨，两条导轨上有 14 个 M4 孔位支持上装载荷便捷安装，双导轨具体尺寸如下

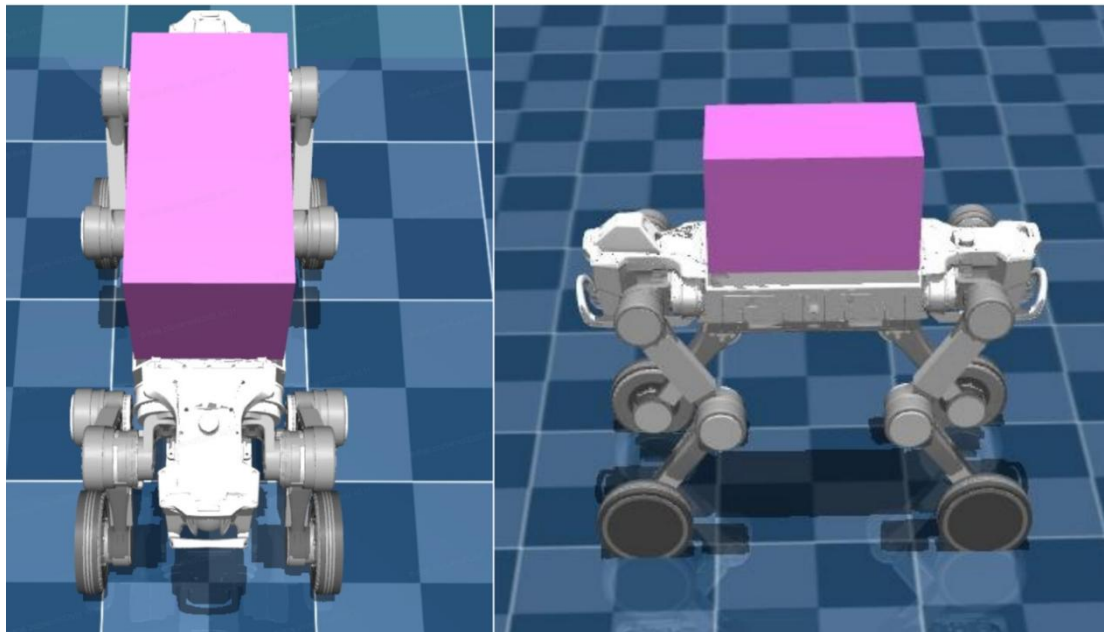


机器人加装负载后，仅支持基础运动模式，请勿在加装负载后使用高台模式

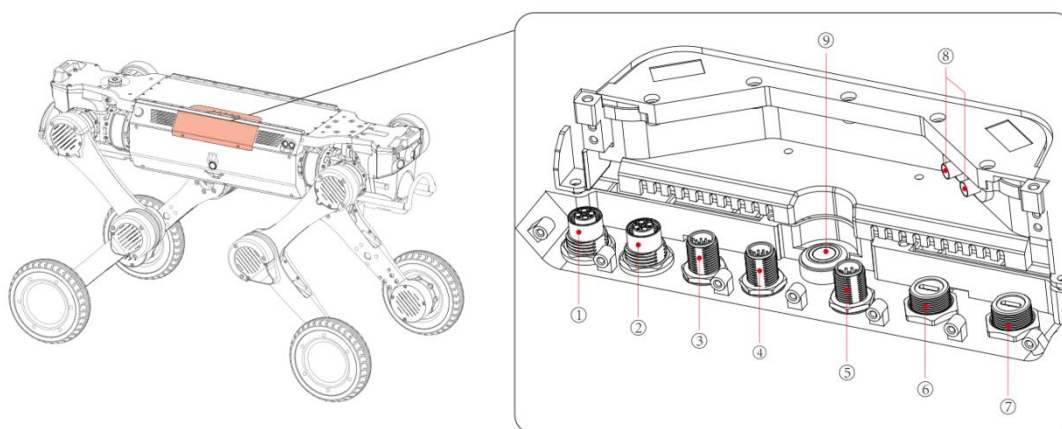
当在机器人背部加装载荷时，机器人的运动性能可能会受影响，请务必在加装前向售后人员咨询相关事宜

1.4.2 重心限制

为避免 D1 Max 机身上加装上装载荷之后，在各类地形运动过程中，腿部与上装载荷发生碰撞干涉，对 D1 Max 的上装空间给出如下推荐示意，具体尺寸为长 40cm，宽 22cm，高 24cm，位置在机身背部居中位置，如下图所示：



1.5 电气拓展接口



序号	接口名称
1	48V(10A)
2	24V(20A)
3	24V(3A)+网口
4	12V(5A)+网口
5	串口 (RS232 x1、RS485 x1、SBUS x2、PPS x1)
6、7	USB3.0 (5V、1A)
8	RTK 天线接口
9	拓展仓按键

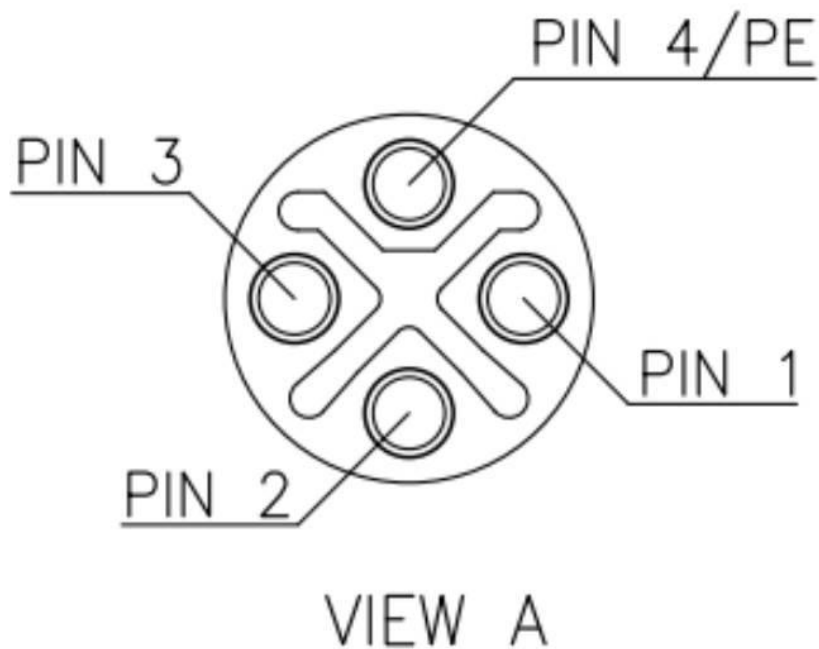
- 所有供电接口的供电总功率在 480W 以内；
- 有航插头配备防护盖，以确保 IP67，不用的航插头务必盖好防护盖；
- 扩展仓按键用于控制扩展接口的供电，在进行上装接线时，务必先关闭扩展仓的电源。

航插扩展功能接口说明：

设备名称	设备路径
RS232	/dev/ttyCH9344USB4
RS485	/dev/ttyCH9344USB5
SBUS1	/dev/ttyCH9344USB2
SBUS2	/dev/ttyCH9344USB3

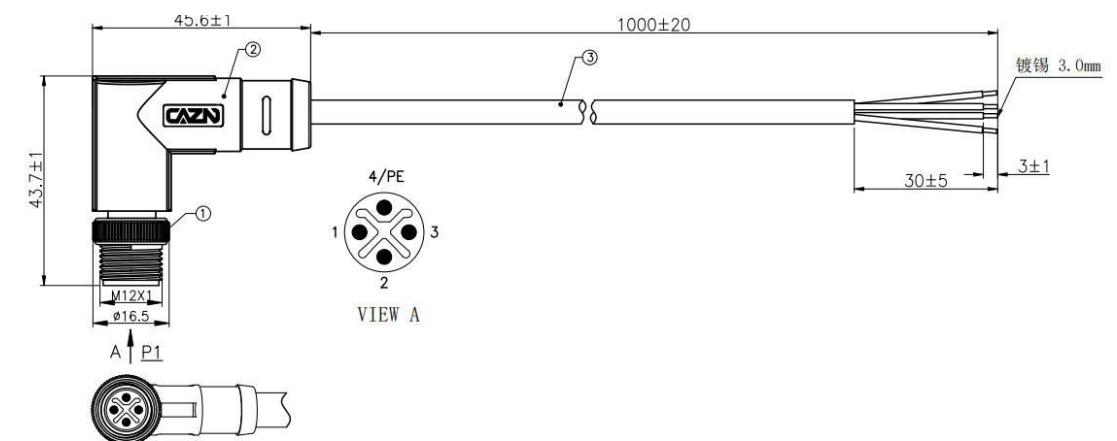
1.5.1 电源口 48V(10A)

- 航插母头



航插名称	48V 10A	
板端航插型号	M12-S4S-GPFM16-135-CS24044	
线序定义	1(棕)	GND
	2(红)	48V
	3(蓝)	48V
	4(黄绿)	GND

- 对插线束



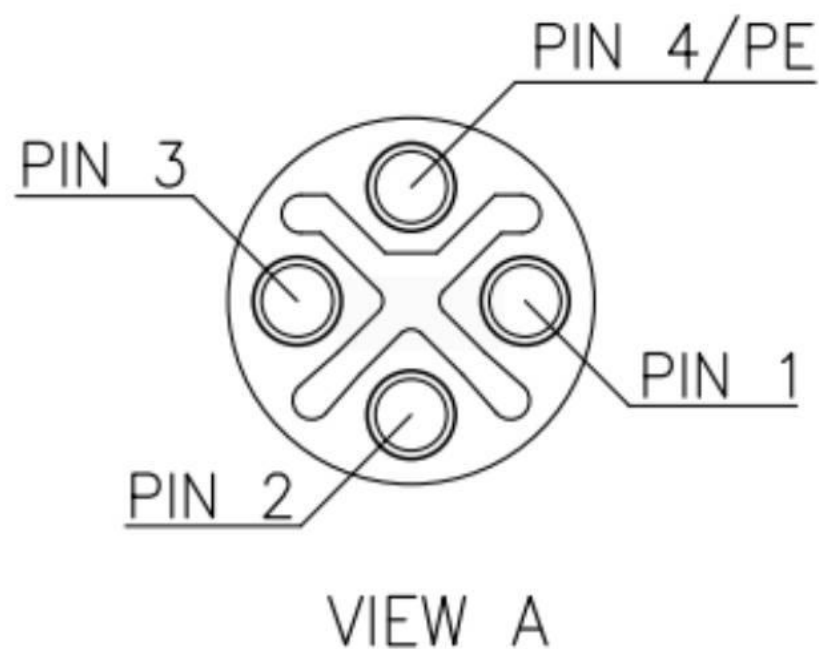
	生产注意事项
s)	
e	
01S	

Wire Diagram 接线定义		
P1	Wire Color 线的颜色	Open 开放
1	Brown 棕	
2	Red 红	
3	Blue 蓝	
4	Yellow Green 黄绿	
Shell 外壳	NA	

3	YY07.04.0150 4C*1.5mm ² (15AWG)+无纺布, 内芯PVC, OD: 2.8外被黑PVC OD:9.0(1.棕; 2.红; 3.黄绿; 4.蓝) 柔性电源电缆 (4C1.5-002)		
2	M12包胶成型 黑色PVC		
1	M12螺纹成型式, 长体胶芯针型插头半成品, 4芯 S编码, 短屏蔽管 1铜大体积螺母 (3+PE款): CS24-M12-P4S-SF01C		
No.	Specification		
GENERAL TOLERANCE (UNLESS SPECIFIED)	绘图 DWN	李平 2025.10.10	STA·思
X. ±0.50 X.X ±0.30 X.XX ±0.20 X.XXX ±0.15	确认 CHK		
	核准	品名	M12-P4S-M

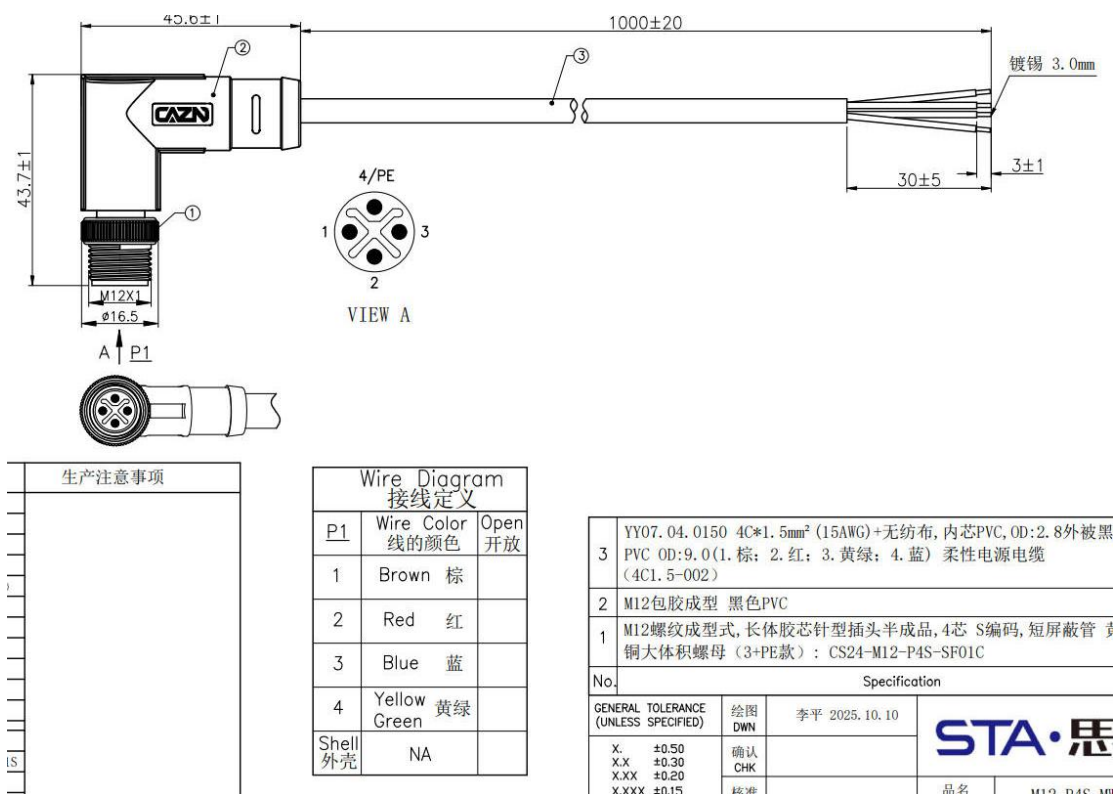
1.5.2 电源口 24V(20A)

- 航插母头



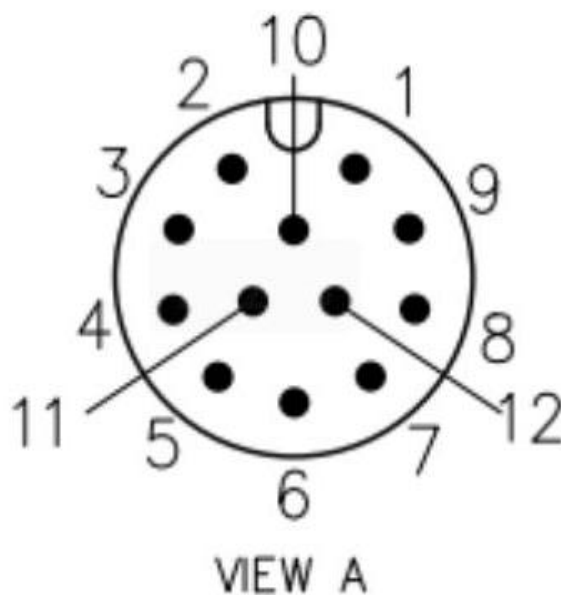
航插名称	24V 20A	
板端航插型号	M12-S4S-GPFM16-135-CS24044	
线序定义	1(棕)	GND
	2(红)	24V
	3(蓝)	24V
	4(黄绿)	GND

- 对插线束



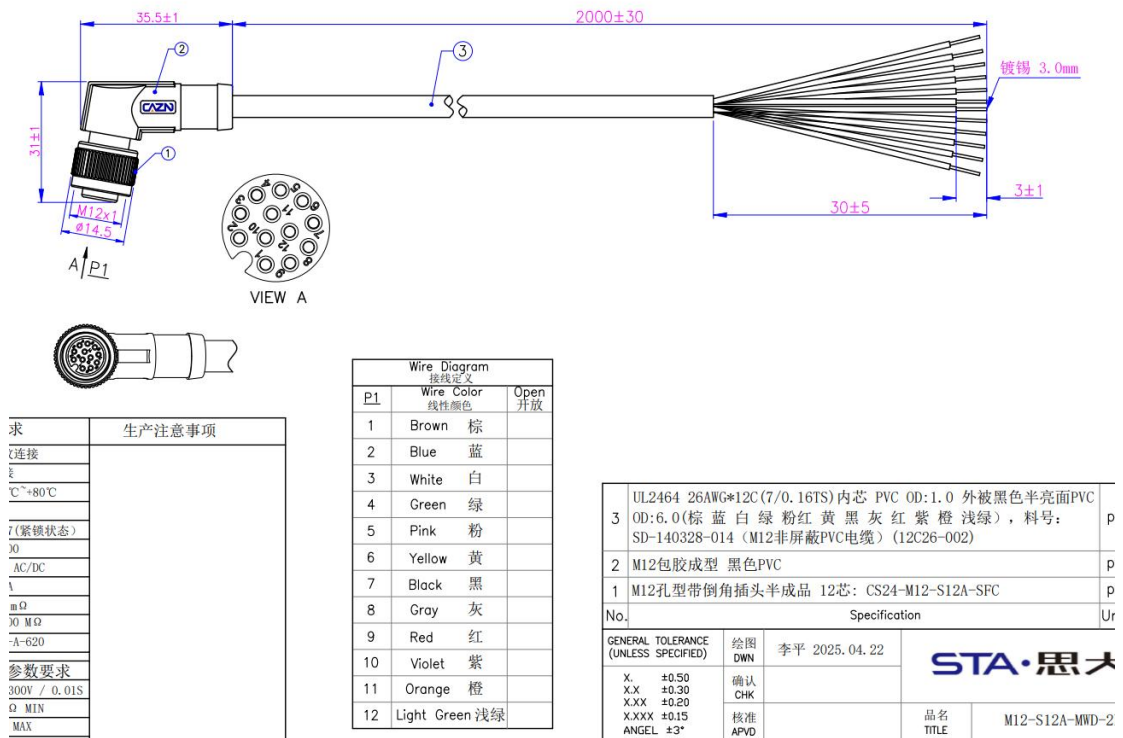
1.5.3 网口(24V)

- 航插母头



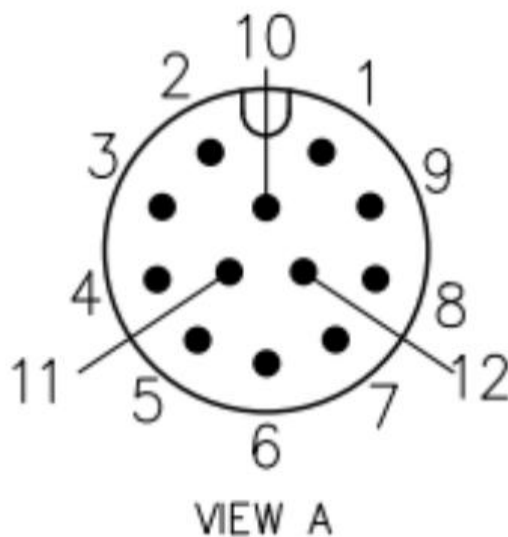
航插名称		网口+24V
板端航插型号		M12- P12A-GPFM12
线序定义	1(棕)	GND
	2(蓝)	24V
	3(白)	24V
	4(绿)	网口 A+
	5(粉)	网口 A-
	6(黄)	网口 B+
	7(黑)	网口 B-
	8(灰)	网口 C+
	9(红)	网口 C-
	10(紫)	GND
	11(橙)	网口 D+
	12(浅绿)	网口 D-

● 对插线束



1.5.4 网口(12V)

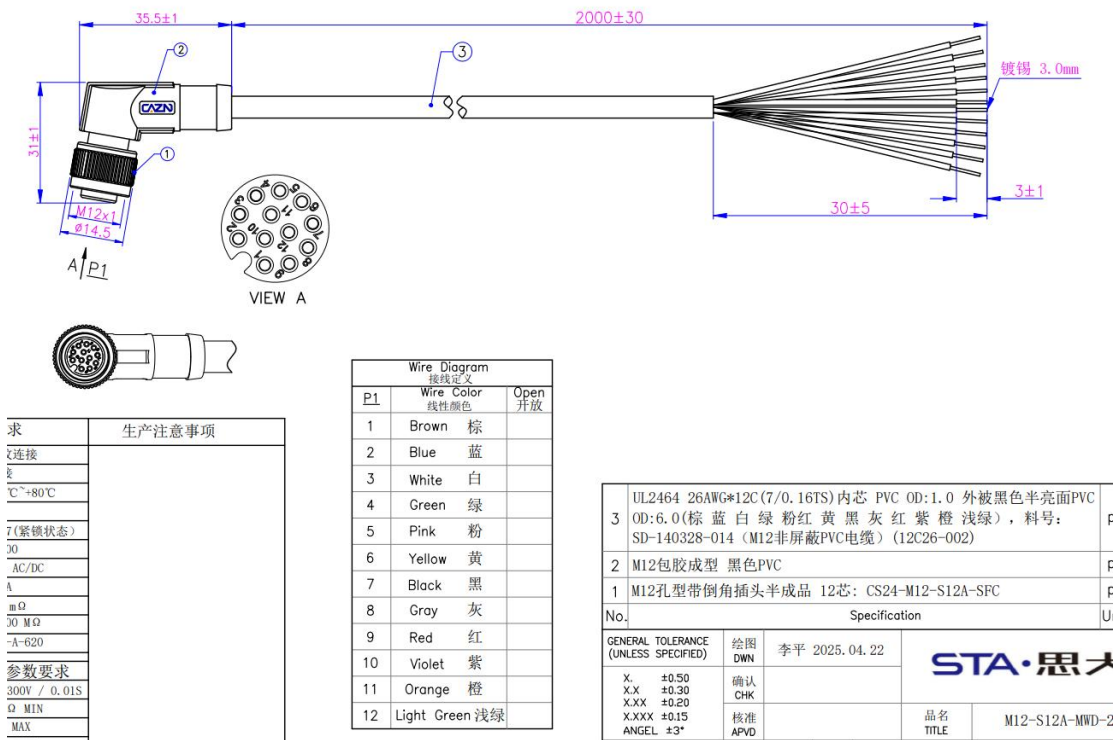
- 航插母头



航插名称	网口+12V	
板端航插型号	M12- P12A-GPFM12	
线序定义	1(棕)	GND
	2(蓝)	12V
	3(白)	12V
	4(绿)	网口 A+
	5(粉)	网口 A-
	6(黄)	网口 B+
	7(黑)	网口 B-
	8(灰)	网口 C+
	9(红)	网口 C-
	10(紫)	GND

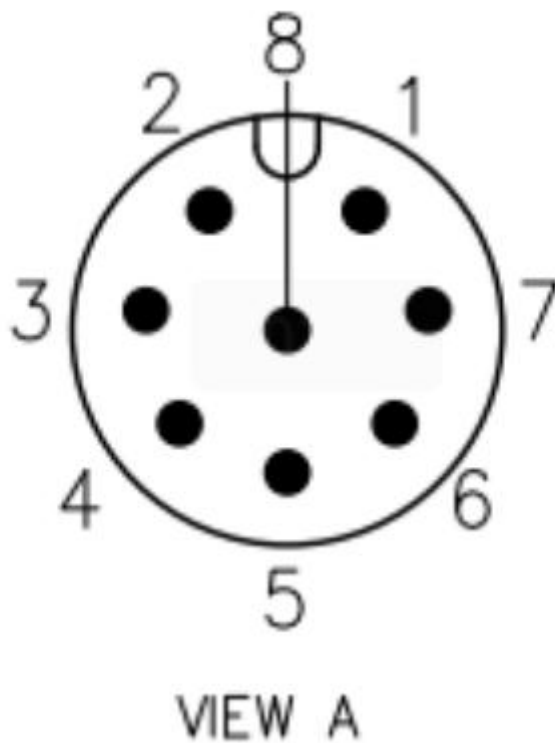
11(橙)	网口 D+
12(浅绿)	网口 D-

● 对插线束



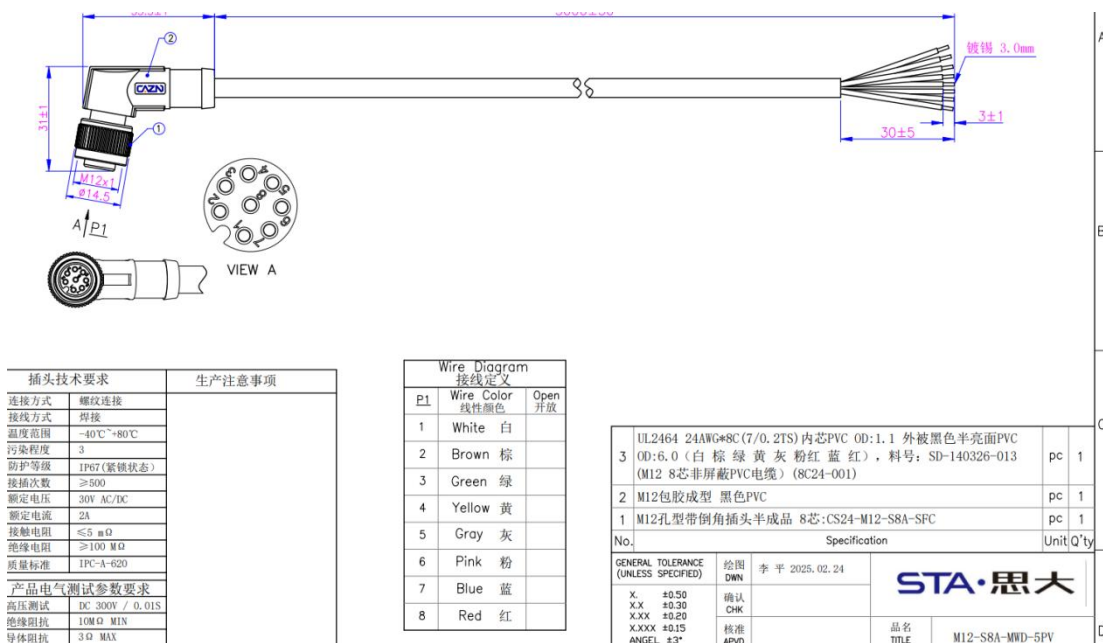
1.5.5 串口

- 航插母头



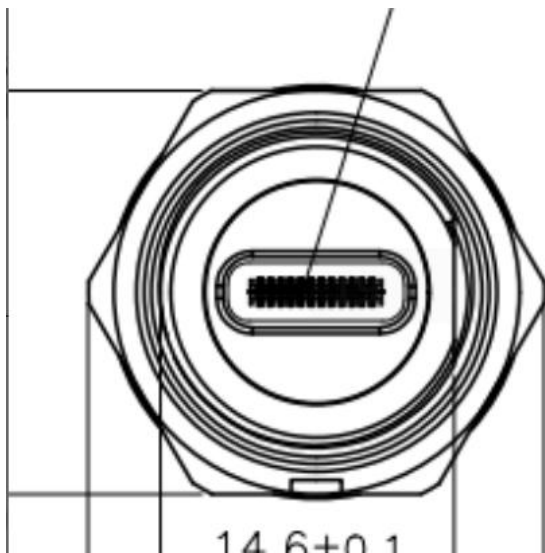
航插名称		串口（RS232、RS485、SBUS*2、PPS）
板端航插型号		M12- P8A-GPFM12
线序定义	1(白)	PPS
	2(棕)	SBUS1
	3(绿)	SBUS2
	4(黄)	RS485A
	5(灰)	RS485B
	6(粉)	RS232_TX
	7(蓝)	RS232_RX
	8(红)	GND

● 对插线束



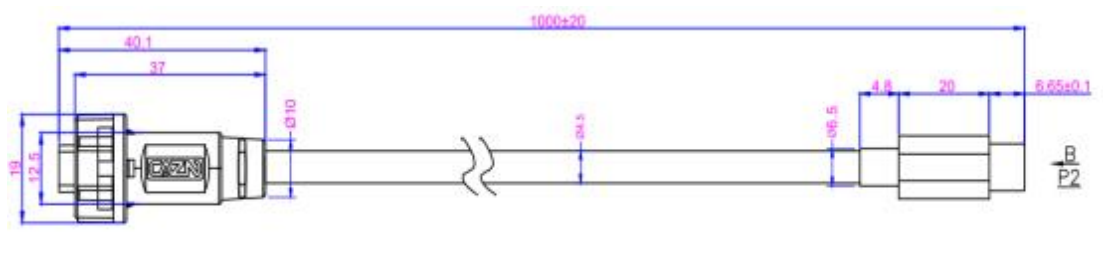
1.5.6 USB 口

● 航插母头

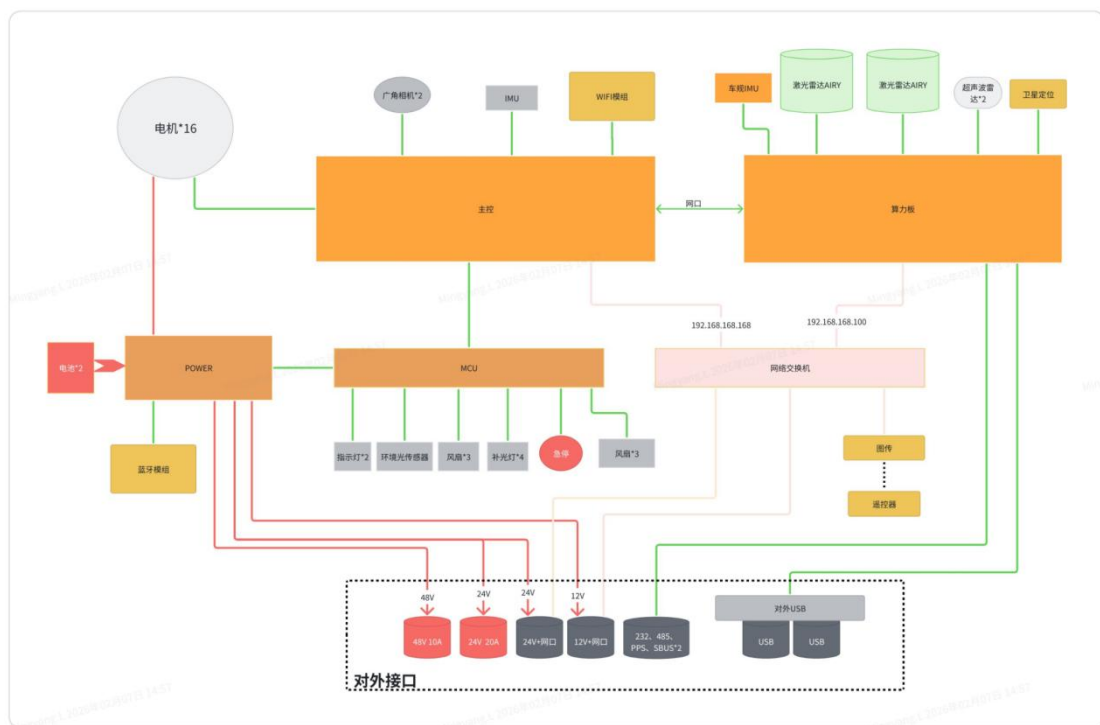


航插名称	USB3.0+5V 两个
板端航插型号	E10B- FT3-PPF
线序定义	无

● 对插线束



1.6 D1 Max 硬件架构图



注：主控板为 RK3588，用来进行四足机器人运动控制，勿在主控板进行开发和部署；
算力板为 Orin NX, 用来进行四足机器人感知控制，可在此算力板进行开发和部署，需做好备份管理，刷机或 OTA 升级可能会被刷新；

二、SDK 开发指南

2.1 SDK 介绍

- 本文档描述了 D1 Max 的软件开发工具包 (SDK)，提供其核心服务接口的详细信息，包括高层运动控制接口、IMU 惯导数据接口、电机状态数据接口。
- 各章节分别介绍不同服务接口的调用方法、数据输入/输出的标准格式定义与一些功能开发示例。
- 接口定义会随量产开发状态产生变更，请以最新版本的接口定义文档为准。
- SDK 程序与四足机器人本体程序通讯存在特定协议，故不同版本的 SDK 程序设备本体程序通讯存在差异，请根据设备版本选择对应版本 SDK 程序。若条件允许，建议更新四足机器人固件到最新版本，以获得更好的兼容性。
- SDK 接口控制时，全部状态均由二次开发者设计指定，本体运控不做额外的干预与状态跳转。
- 运行 SDK 时，请保证系统有足够的资源，否则可能出现运动控制模块失效的情况。

2.2 SDK 软件服务接口列表

序号	接口名称	功能	SDK 版本
1	高层运动控制接口	实现机器狗的高层运动控制，包括前后转向速度运动，趴下，旋转，四肢站立等	V0.0.2
2	视频流推送数据接口	向客户端推送相机本体视频流	V0.0.2
3	设备状态服务接口	查询并获取当前设备状态和系统资源使用信息	V0.0.3
4	惯导状态数据接口	获取 IMU 原始数据信息	V0.0.4
5	超声波雷达接口	实现自主避障功能	v0.0.5
6	遥控器和 sdk 控制权切换功能及 sdk 接口	实现控制权切换功能	v0.0.6
7	RTK 模块的 sdk 接口	实现自主定位功能	v0.0.6
8	自主回充功能及 API	实现自主充电功能	v0.0.9

	接口(选配)		
9	slam 建图导航功能及 API 接口 (选配)	实现自主导航功能	v0.1.0

2.3 安全保护机制

2.3.1 硬急停

- 触发方式：按下机身急停按钮
- 效果：机器狗缓慢着地，红灯亮起
- 指示灯状态
 - 按下时：红灯亮
 - 松开后：绿灯亮
- 恢复：旋开急停按钮后，系统自动恢复(若无其他故障)

2.3.2 软急停

- 触发方式：通过遥控器触发或者通过 sdk 接口触发
- 效果：机器人立即停车，不再响应任何控制指令
- 恢复：接触软急停后可恢复正常操作

从 V0.0.6 版本 sdk 起，支持遥控器和 sdk 控制权切换功能，若要实现在 sdk 控制阶段通过遥控器触发设备软急停，必须得在遥控器中打开 app 获取到控制权后，然后便可以随时使用遥控器右上方红色急停按钮触发软急停，触发软急停后，机器狗显示红灯，保持当前姿态，不再响应任何控制指令，再次点击遥控器急停按键即可恢复急停。

2.3.3 内部异常保护(自动触发)

以下情况将触发内部保护机制，机器人自动停车并缓慢着地

- 遥控信号断连(含网络阻塞)，重新连接 APP 即可恢复
- 电池在位但电量<10%
- 关节故障
- IMU(惯性测量单元)通信中断

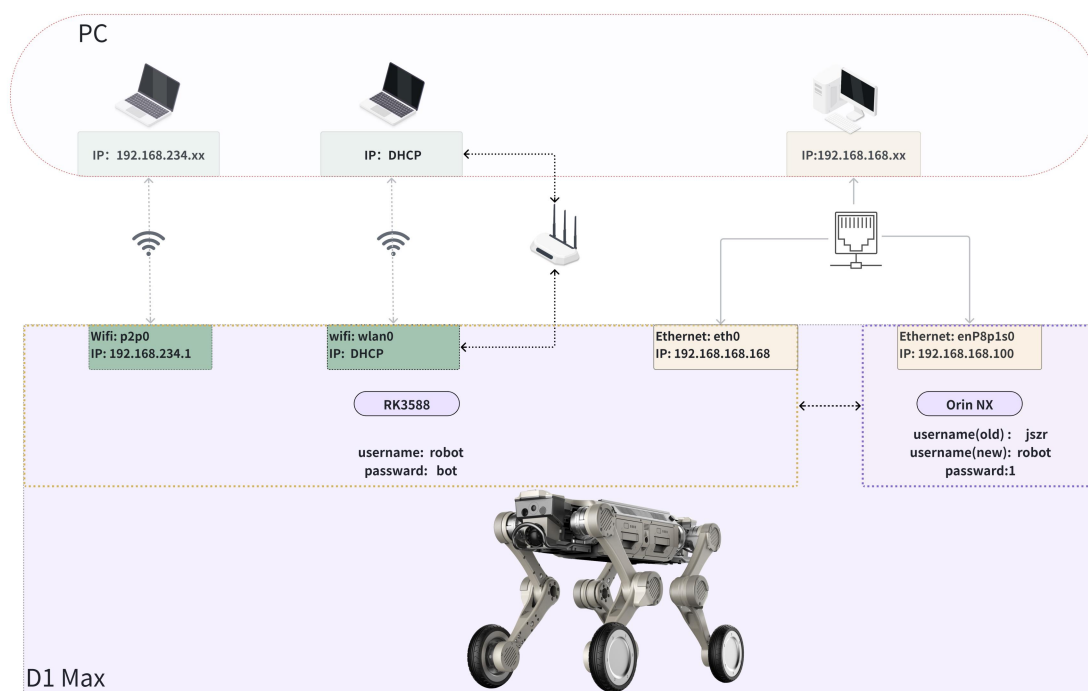
2.4 环境依赖

- 操作系统: Ubuntu 22.04
- 构建工具: CMake 3.8+, GCC 11+
- 依赖库: Boost 1.74+
- 编程语言: C++

2.5 设备登录

网络连接

- Wifi 连接
 - wifi 名称: XG2WIFI_XXXXXX
 - wifi 密码: 12345678
- 有线连接
 - 使用网线连接机器狗拓展网口和控制端 PC
 - 设置控制端 PC 网口 IP 为 192.168.168.x(不能设置为 192.168.168.168 和 192.168.168.100)



ssh 登录

Bash

#通过 wifi 连接

ssh robot@192.168.234.1 #RK3588, password: bot

#通过网线连接

ssh robot@192.168.168.168 #RK3588, password: bot

ssh robot@192.168.168.100 #Orin NX,password: 1

2.6 SDK 获取

way1: 通过 ssh 进入机器狗本体，查询当前的固件版本信息后，将查询到的四足机器人固件版本号截图发送售后技术支持团队获取对应的 SDK 安装包与示例代码，查询方式如下：

bash

```
ssh robot@192.168.234.1      #password: bot
cat /opt/release/version.yaml #查看 rk 对应固件版本

ssh robot@192.168.168.100    #password: 1
cat /opt/release/version.yaml #查看 nx 对应固件版本
```

way2:通过如下 github 地址获取

bash

```
git clone https://github.com/AgibotTech/Agibot_D1_Max.git
```

2.7 Demo 运行

在本地 PC 端，进入到 sdk 文件中

Bash

```
# 进入 C++ 演示目录
cd example

# 创建构建目录
mkdir build && cd build

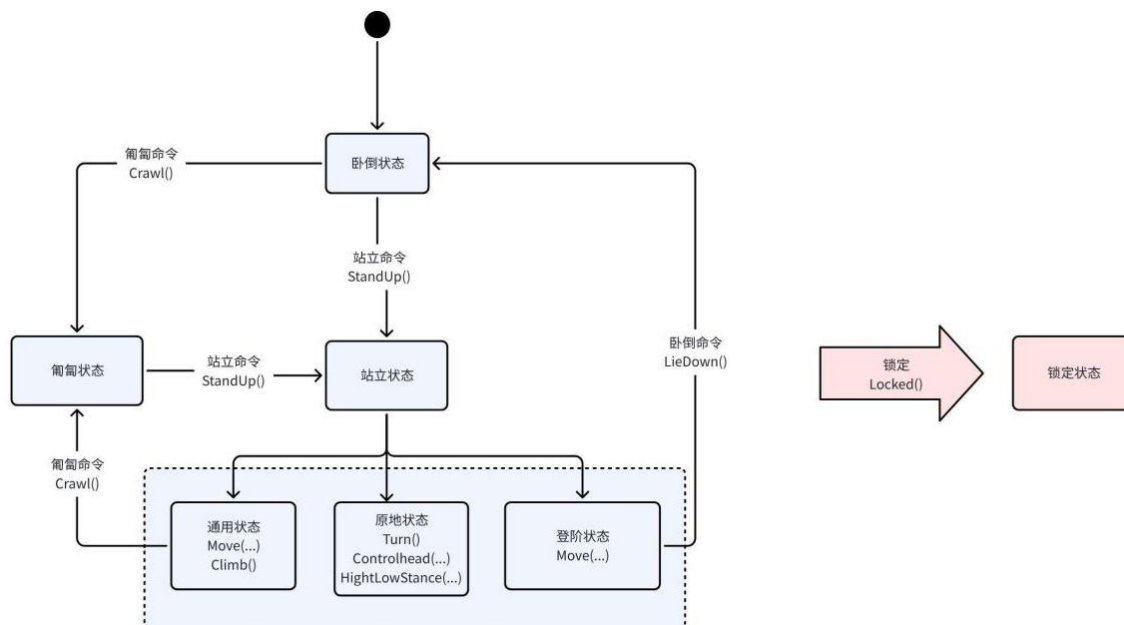
#配置并编译项目
cmake ..
make -j6

#运行 demo 示例
./example_data 192.168.168.168 8081 #有线连接
./example_data 192.168.234.1 8081  #无线连接
```

在运行 demo 文件前，务必使遥控器 app 退出后台程序

2.8 SDK 控制说明

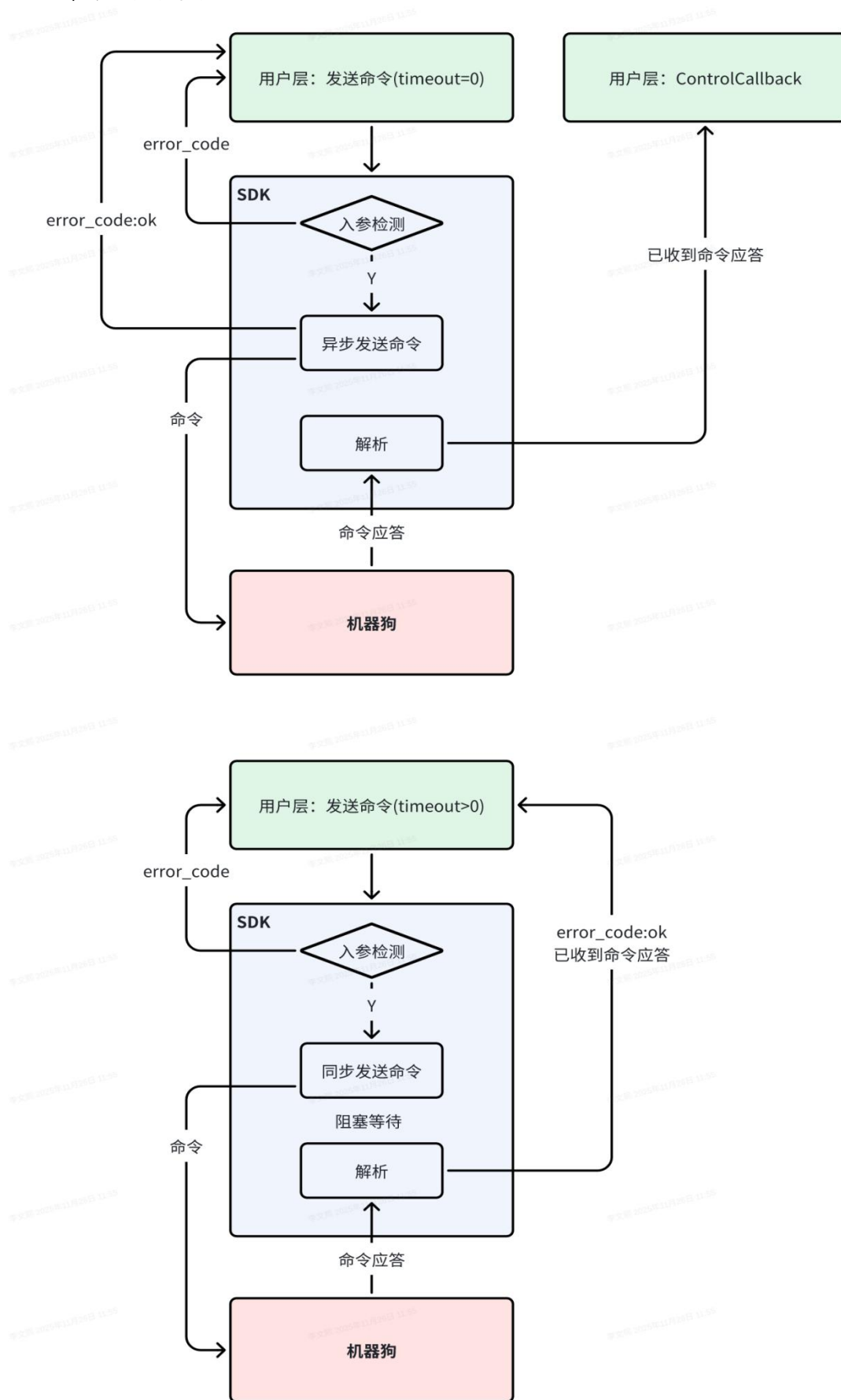
- sdk 指令下发需要按照以下状态跳转逻辑，否则可能会造成机器摔倒、故障或不响应等异常情况。



注意：

1. 当下发站立命令时，完成站立后会根据当前的模式自动切换到通用状态或者原地状态
2. 锁定状态下切到任意其他状态下就算解锁

● SDK 命令下发流程如下



2.9 关节命名规范

为便于识别与调试，本设备采用如下命名规则：

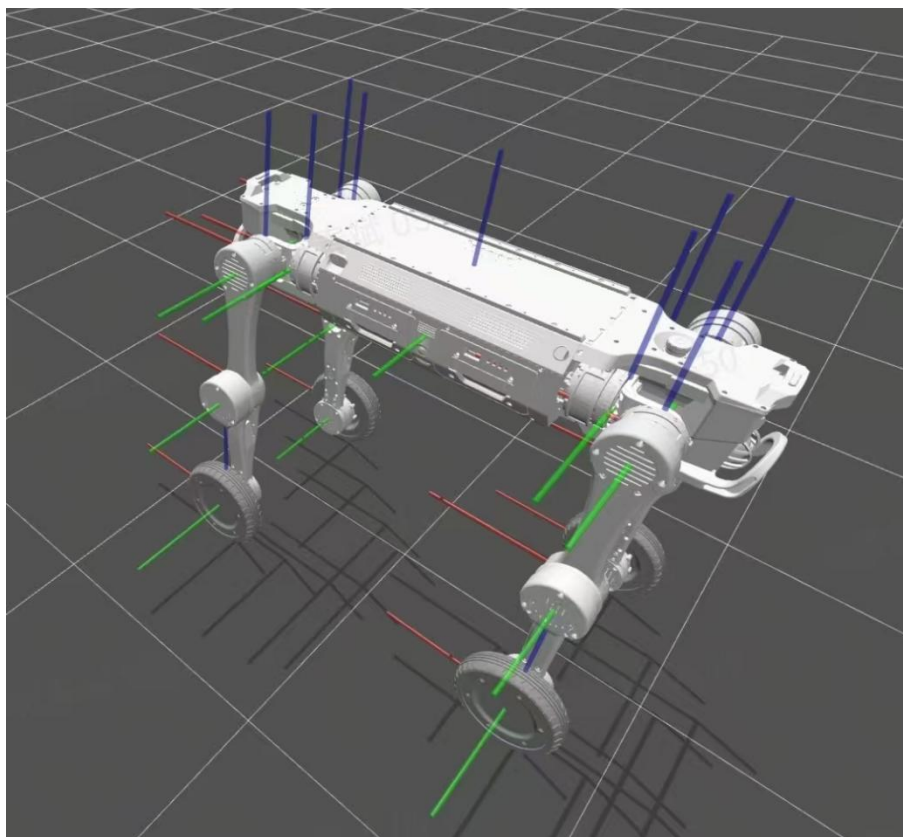
- 方向缩写
 - f = front(前)
 - b = back(后)
 - l = left(左)
 - r = right(右)
- 编号含义
 - 1 = 髋部滚转(hip_roll)
 - 2 = 髋部俯仰(hip_pitch)
 - 3 = 膝部俯仰(knee_pitch)
 - 4 = 足端(foot)
- 示例命名
 - fr1_hip_roll: 右前腿 1 号关节(髋部滚转)
 - bl3_knee_pitch: 左后腿 3 号关节(膝部俯仰)

front_left_leg	#左前腿
front_right_leg	#右前腿
back_left_leg	#左后腿
back_right_leg	#右后腿
fl1_hip_roll	#左前 1 号
fl2_hip_pitch	#左前 2 号
fl3_knee_pitch	#左前 3 号
fl4_foot	#左前 4 号
fr1_hip_roll	#右前 1 号
fr2_hip_pitch	#右前 2 号
fr3_knee_pitch	#右前 3 号
fr4_foot	#右前 4 号
bl1_hip_roll	#左后 1 号
bl2_hip_pitch	#左后 2 号
bl3_knee_pitch	#左后 3 号
bl4_foot	#左后 4 号
br1_hip_roll	#右后 1 号
br2_hip_pitch	#右后 2 号
br3_knee_pitch	#右后 3 号
br4_foot	#右后 4 号

2.10 传感设备坐标定义

传感器名称	传感器坐标（相对机身 BASE 原点） （单位 mm）			备注
	x	y	z	
IMU_ICM42688（运控用）	0	0	36.2	
IMU_LUA300C（车规级）	0	0	56.9	
前激光雷达 AIRY	404.3	0	-37.7	详见规格书
前摄像头 IMX415	412.3	0	37.8	详见规格书
右前补光灯	420.3	-30.9	28.3	照射范围 120°
左前补光灯	420.3	30.9	28.3	照射范围 120°
后激光雷达 AIRY	-404.3	0	-37.7	详见规格书
后摄像头 IMX415	-412.3	0	37.8	详见规格书
右后补光灯	-420.3	-30.9	28.3	照射范围 120°
左后补光灯	-420.3	30.9	28.3	照射范围 120°
右前超声波雷达 RADAR A22	179.2	-100.2	50	详见规格书
左前超声波雷达 RADAR A22	179.2	100.2	50	详见规格书

当各个关节均为零度时，各坐标系如下图。红色为 x 轴，绿色为 y 轴，蓝色为 z 轴。
关节的旋转轴和旋转正方向请参考下图



三、SDK 函数介绍

- `SDKClient` 是 Agibot D1 Max 机器人 SDK 的核心通信类，用于与机器人建立连接、断开连接、发送控制命令、配置上报数据等。开发者通过该类可实现机器人运动控制、模式切换、急停、安全锁定等多种功能。
- 该类支持**阻塞模式**与**非阻塞模式**两种调用方式
 - 阻塞模式：函数会等待操作完成或超时后返回结果。
 - 非阻塞模式：函数立即返回，并通过回调函数通知结果。
- SDK 目录结构

sdk 目录结构

```
├── example/          # 演示应用程序 (C++)
├── docs/             # 详细的文档文件
├── include/          # SDK 的 C++ 头文件
└── lib/              # 针对不同架构的预编译库文件 (.so)
```

- 当前版本目前开放了 HighLevel 高层控制接口，后续会逐步开放更多 sdk 接口

3.1 HighLevel API 函数介绍

3.1.1 错误码机制

- 异常码
 - `error_code`
 - ◆ 成功: `ec.value() == 0`
 - ◆ 失败: `ec.value() != 0`
 - `error_msg`: 可通过 `ec.message()` 获取详细的错误描述。
 - 示例

C++

```
std::error_code ec = sdk.Connect("192.168.234.1", 8081);
if (ec) {
    std::cerr << "连接失败: " << ec.message() << std::endl;
} else {
    std::cout << "连接成功" << std::endl;
}
```

- 错误码定义

目前所有错误码均采用 C++ 标准中定义

SDK 中常用的错误码表如下

枚举值	整数值	含义
success	0	成功
invalid argument	22	参数无效
already connected	106	已连接
not connected	107	未连接
timeout	110	操作超时
connection refused	111	连接拒绝
connection already in progress	114	连接执行中
operation in progress	115	正在执行中
operation canceled	125	操作被取消

● 示例

C++

```
std::error_code ec = sdk.Connect("192.168.234.1", 8081);
if (ec) {
    std::cerr << "连接失败: " << ec.message() << std::endl;
} else {
    std::cout << "连接成功" << std::endl;
}
```

3.1.2 类型定义

类型名	定义	说明
ConnectHandler	<code>std::function<void(const std::error_code&)></code>	连接完成后的回调函数
DisConnectHandler	<code>std::function<void(const std::error_code&)></code>	断开完成后的回调函数
ErrorHandler	<code>std::function<void(const</code>	SDK 内部错误回

`std::error_code&)>`

调函数

3.1.3 构造与析构

● 构造

函数名	<code>SDKClient(ErrorHandler error_callback = [] (const std::error_code&) {})</code>
功能概述	构造一个 SDK 客户端对象
参数	<code>error_callback</code> : SDK 内部通信异常回调函数
返回值	
备注	

● 析构

函数名	<code>~SDKClient()</code>
功能概述	析构函数，释放资源
参数	
返回值	
备注	

3.1.4 连接管理接口

● 建立连接

函数名	<code>std::error_code Connect(std::string ip, std::string port, bool block = false, ConnectHandler handler = {})</code>
功能概述	与机器狗建立通信连接
参数	<code>ip</code> : 目标 IP 地址 <code>port</code> : 目标端口号 <code>block</code> : 是否阻塞: <code>true</code> 阻塞模式; <code>false</code> 非阻塞 <code>handler</code> : 非阻塞模式下连接完成后的回调函数
返回值	成功: 返回 <code>std::error_code()</code> 失败: 返回带错误信息的 <code>std::error_code</code>
备注	通信建立失败时，终端输出失败日志

● 断开链接

函数名	<code>std::error_code Disconnect(bool block = false, DisConnectHandler handler = {})</code>
功能概述	断开与机器人的连接
参数	<code>block</code> : 是否阻塞模式: <code>true</code> 阻塞模式; <code>false</code> 非阻塞 <code>handler</code> : 非阻塞模式下断开完成后的回调函数
返回值	成功: 返回 <code>std::error_code()</code> 失败: 返回带错误信息的 <code>std::error_code</code>
备注	通信建立失败时，终端输出失败日志

- 获取连接状态

函数名	bool IsConnected() const
功能概述	判断当前是否已连接机器人
参数	无
返回值	true: 已连接 false: 未连接
备注	

3.1.5 回调设置函数

设置控制应答回调函数

函数名	void SetControlCallback(std::shared_ptr<IControlCallback>& control_callback)
功能概述	设置控制应答回调函数，用于非阻塞模式下接收命令反馈
回调函数	详见 IControlCallback 类的详细说明
备注	

IControlCallback 类

- 说明：控制命令回调接口，该接口用于接收机器人确认“已收到”控制命令的回调

仅在非阻塞模式下使用

- 定义

```

C++
IControlCallback
class ROBOT_EXPORT_API IControlCallback {
public:
    virtual void OnSoftEmergencyStop(bool on) {}
    virtual void OnStandUp() {}
    virtual void OnLieDown() {}
    virtual void OnCrawl() {}
    virtual void OnClimb() {}
    virtual void OnSlim() {}
    virtual void OnGait() {}
    virtual void OnReverseHeadTail() {}
    virtual void OnMode(int mode) {}
    virtual void OnSpeed(int speed_level) {}
    virtual void OnLocked() {}
    virtual void OnFrontLight(bool on) {}
    virtual void OnBackLight(bool on) {}
    virtual void OnAutoModeLight(bool on) {}
    virtual void OnLuxConfig(bool on) {}
    virtual void OnImuConfig(int freq) {}
    virtual void OnMcConfig(bool on) {}
    virtual void OnTakeControlAck(const TakeControlAck& ack) {}
    virtual void OnReleaseControlAck(const ReleaseControlAck& ack) {}
};
virtual ~IControlCallback() = default;
};
    
```

回调列表

回调函数	说明	参数
<code>OnSoftEmergencyStop(bool on)</code>	已收到急停命令	<code>on = true</code> 开启急停 <code>on = false</code> 关闭急停
<code>OnStandUp()</code>	已收到站立命令	—
<code>OnLieDown()</code>	已收到卧倒命令	—
<code>OnCrawl()</code>	已收到匍匐命令	—
<code>OnClimb()</code>	已收到爬高台命令	—
<code>OnSlim()</code>	已收到瘦身命令	—
<code>OnGait()</code>	已收到步态命令	—
<code>OnReverseHeadTail()</code>	已收到调转头尾命令	—
<code>OnMode(int mode)</code>	已收到模式切换命令	模式编号
<code>OnSpeed(int speed_level)</code>	已收到速度切换命令	速度档位
<code>OnLocked()</code>	已收到锁定命令	-
<code>OnFrontLight(bool on)</code>	已收到前补光灯命令	<code>true</code> 开启, <code>false</code> 关闭
<code>OnBackLight(bool on)</code>	已收到后补光灯命令	<code>true</code> 开启, <code>false</code> 关闭
<code>OnLuxConfig(bool on)</code>	已收到光强值配置命令	<code>true</code> 开启, <code>false</code> 关闭
<code>OnImuConfig(int freq)</code>	已收到 IMU 配置命令	频率值
<code>OnMcConfig(bool on)</code>	已收到运控数据配置命令	<code>true</code> 开启, <code>false</code> 关闭

OnTakeControlAck(const TakeControlAck& ack)	已收到获取控制权命令应答	应答信息
OnReleaseControlAck(const ReleaseControlAck& ack)	已收到释放控制权命令应答	应答信息

- 示例

```
C++

#include "robot_sdk/sdk_callback.hpp"

class MyDataCallback : public robot_sdk::IDataCallback {
public:
    void OnImuData(const ImuData& data) override {
        // 轻量处理，例如缓存
    }
};

class MyControlCallback : public robot_sdk::IControlCallback {
public:
    void OnStandUp() override {
        // 收到站立命令应答
    }
};
```

- 注意事项

不要在回调中执行耗时任务。

若需要耗时处理，请在回调中将数据复制并投递至独立线程。

设置数据接收回调函数

函数名	void SetDataCallback(std::shared_ptr<IDataCallback>& data_callback)
功能概述	设置数据接收回调函数，当机器人数据上报时触发
回调函数	详见 IDataCallback 类的详细说明
备注	

IDataCallback 类

- 说明：数据上报回调接口，用户需根据需要进行实现此接口以接收来自机器人系统的数据

回调函数必须**轻量级**，不能执行耗时操作。

在回调函数中仅建议进行数据复制、校验等快速操作。

若需执行耗时任务（如数据库写入、文件 I/O、复杂计算、网络发送等），请在回调函数内部将数据复制到独立线程或线程池中处理。

● 定义

```
C++
IDataCallback
class ROBOT_EXPORT_API IDataCallback {
public:
    virtual void OnImuData(const ImuData& data) {}
    virtual void OnLuxData(const LuxData& data) {}
    virtual void OnMcData(const MotionData& data) {}
    virtual void OnRobotStateData(const RobotState& data) {}
    virtual void OnFaultData(const FaultDatas& data) {}
    virtual void OnControlLost(const ControlLostInfo& info) {}
    virtual void OnControlAvailable(const ControlAvailableInfo& info) {}

    virtual ~IDataCallback() = default;
};
```

● 回调列表

回调函数	描述	参数	调用频率
OnImuData	IMU 数据回调	<code>const ImuData& data</code> : IMU 数据结构	按配置频率上报
OnLuxData	光照强度数据回调	<code>const LuxData& data</code> : 光照强度数据结构	配置后定频 1Hz 上报
OnRobotStateData	机器人状态数据回调	<code>const RobotState& data</code> : 机器人状态结构	1Hz 主动上报
OnFaultData	故障信息数据回调	<code>const FaultDatas& data</code> : 故障数据集合	发生故障时主动上报
OnMcData	机器人运动数据回调	<code>const MotionData& data</code> : 机器人状态结构	配置后定频 100Hz 上报
OnControlLost	控制权丢失回调	<code>const ControlLostInfo& info</code> : 控制权丢失信息	
OnControlAvailable	控制权可用回调	<code>const ControlAvailableInfo& info</code> : 控制权可用信息	

3.1.6 控制接口

3.1.6.1 急停控制(SoftEmergencyStop)

函数名	std::error_code SoftEmergencyStop(bool on, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})
功能概述	急停控制。软急停后，系统不会响应其他运动命令,并且速度将为 0
参数	on: 急停状态: true:急停; false:解除急停 timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码: - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	- 异步模式: 函数返回仅表示命令已发送, 发送结果通过回调函数通知 - 同步模式: 函数返回即表示命令发送结果 启用急停后, 系统将不响应其他运动命令, 此时速度为 0

3.1.6.2 站立(StandUp)

函数名	std::error_code StandUp(int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})
功能概述	机器人站立动作
参数	timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码: - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	调用站立后, 站立过程中会上报 'MOTION_STATUS_STAND_UP' 状态, 站立完成后会根据模式自动进入 'MOTION_STATUS_GENERAL' 或 'MOTION_STATUS_IN_PLACE' 状态。

3.1.6.3 卧倒(LieDown)

函数名	std::error_code LieDown(int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})
功能概述	机器人卧倒动作
参数	timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码: - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	

3.1.6.4 匍匐(Crawl)

函数名	<code>std::error_code Crawl(int timeout_ms = 0, WriteHandler handler = [](const std::error_code&, std::size_t) {})</code>
功能概述	机器人匍匐动作
参数	timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	

3.1.6.5 爬高台(Climb)

函数名	<code>std::error_code Climb(int timeout_ms = 0, WriteHandler handler = [](const std::error_code&, std::size_t) {})</code>
功能概述	机器人爬高台动作
参数	timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	仅在"通用模式"下使用

3.1.6.6 瘦身(Slim)

函数名	<code>std::error_code Slim(int timeout_ms = 0, WriteHandler handler = [](const std::error_code&, std::size_t) {})</code>
功能概述	机器人瘦身动作
参数	timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	仅在"通用模式"下使用

3.1.6.7 步态(Gait)

函数名	<code>std::error_code Gait(int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	机器人步态姿态
参数	<code>timeout_ms</code> : 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) <code>handler</code> : 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	仅在"通用模式"下使用

3.1.6.8 过挡鼠板姿态(DSB)

函数名	<code>std::error_code DSB(int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	过挡鼠板姿态
参数	<code>timeout_ms</code> : 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) <code>handler</code> : 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	仅在"通用模式"下使用

3.1.6.9 调转头尾方向(ReverseHeadTail)

函数名	<code>std::error_code ReverseHeadTail(int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	调转机器人头尾方向
参数	<code>timeout_ms</code> : 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) <code>handler</code> : 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	

3.1.6.10 设置当前运行模式(SetMode)

函数名	<code>std::error_code SetMode(int mode, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	设置机器人运行模式，默认为通用模式。
参数	mode: 1 为通用模式; 2 为原地模式; 3 为登阶模式 timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- `std::errc::success`: 操作成功 - 常见失败错误码: - `std::errc::timed_out`: 操作超时 - `std::errc::not_connected`: 未连接 - `std::errc::operation_canceled`: 操作已取消
备注	

3.1.6.11 设置运动速度等级(SetSpeed)

函数名	<code>std::error_code SetSpeed(int speed_level, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	设置速度等级，主要影响通用模式下 `Move` 命令的速度，默认为低速。 低速对 Move 接口的影响： forward_back: [-1.0, 1.0]对应[-1.0m/s, +1.0m/s] left_right: [-1.0, 1.0]对应[-0.5m/s, +0.5m/s] yaw: [-1.0, 1.0]对应[-1.5rad/s, +1.5rad/s] ===== 中速对 Move 接口的影响： forward_back: [-1.0, 1.0]对应[-2.0m/s, +2.0m/s] left_right: 当 -1.0m/s < forward_back < 1.0m/s 时, [-1.0, 1.0]对应[-0.5m/s, +0.5m/s] 当 forward_back > 1.0m/s 或者 forward_back < -1.0m/s 时, 限制为 0m/s yaw: 当 -1.0m/s < forward_back < 1.0m/s 时, [-1.0, 1.0]对应[-1.5rad/s, +1.5rad/s] 当 forward_back > 1.0m/s 或者 forward_back < -1.0m/s 时, [-1.0, 1.0]对应[-1.0rad/s, +1.0rad/s] ===== 高速对 Move 接口的影响： forward_back: [-1.0, 1.0]对应[-3.0m/s, +3.0m/s] left_right: 当 -1.0m/s < forward_back < 1.0m/s 时, [-1.0, 1.0]对应[-0.5m/s, +0.5m/s] 当 forward_back > 1.0m/s 或者 forward_back < -1.0m/s 时, 限制为 0m/s yaw: 当 -1.0m/s < forward_back < 1.0m/s 时, [-1.0, 1.0]对应[-1.5rad/s, +1.5rad/s] 当 forward_back > 1.0m/s 或者 forward_back < -1.0m/s 时, [-1.0, 1.0]对应[-1.0rad/s, +1.0rad/s] 当 forward_back > 2.0m/s 或者 forward_back < -2.0m/s 时, [-1.0, 1.0]对应[-0.5rad/s, +0.5rad/s]
参数	speed_level: 1 为低速; 2 为中速; 3 为高速 timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- `std::errc::success`: 操作成功 - 常见失败错误码: - `std::errc::timed_out`: 操作超时 - `std::errc::not_connected`: 未连接 - `std::errc::operation_canceled`: 操作已取消
备注	

3.1.6.12 锁定机器人(Locked)

函数名	std::error_code Locked(int timeout_ms = 0, WriteHandler handler = [])(const std::error_code&, std::size_t {})
功能概述	锁定机器人：机器锁在当前固定的位置，各个关节保持不动。
参数	timeout_ms: 0 表示异步模式； >0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数； 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码： - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	注意：当下发切换到其他运动状态的命令（站立、匍匐、卧倒）时，自动解锁，切换到其他动作状态

3.1.6.13 前补光灯控制(FrontLight)

函数名	std::error_code FrontLight(bool on, int timeout_ms = 0, WriteHandler handler = [])(const std::error_code&, std::size_t {})
功能概述	设置前补光灯，设置后自动模式关闭。
参数	on: true 打开； false 关闭 timeout_ms: 0 表示异步模式； >0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数； 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码： - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	

3.1.6.14 后补光灯控制(BackLight)

函数名	std::error_code BackLight(bool on, int timeout_ms = 0, WriteHandler handler = [])(const std::error_code&, std::size_t {})
功能概述	后补光灯控制
参数	on: true 打开； false 关闭 timeout_ms: 0 表示异步模式； >0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数； 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码： - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	

3.1.6.15 自动补光灯模式(AutoModeLight)

函数名	std::error_code AutoModeLight(bool on, int timeout_ms = 0, WriteHandler handler = [])(const std::error_code&, std::size_t {})
功能概述	设置自动补光灯模式
参数	on: true 打开； false 关闭 timeout_ms: 0 表示异步模式； >0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数； 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码： - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	V0.0.6 版本 sdk 新增

3.1.7 运动与姿态控制接口

3.1.6.16 移动(Move)

函数名	<code>std::error_code Move(float left_right, float forward_back, float yaw, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	控制机器狗进行前后左右移动及旋转，用在通用模式下进行移动，单位：百分比，根据 SpeedLevel 来限速。最新的 Move 指令会维持 1s。
参数	left_right: 左右平移速度，范围 [-1.0, 1.0]；正数左移，负数右移 forward_back: 前后移动速度，范围 [-1.0, 1.0]；正数前进，负数后退 yaw: 旋转速度，范围 [-1.0, 1.0]；正数左转，负数右转 timeout_ms: 0 表示异步模式；>0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数；同步模式下不使用
返回值	- `std::errc::success`: 操作成功 - 常见失败错误码： - `std::errc::timed_out`: 操作超时 - `std::errc::not_connected`: 未连接 - `std::errc::operation_canceled`: 操作已取消 - `std::errc::invalid_argument`: 参数无效
备注	异步模式：函数返回仅表示命令已发送，发送结果通过回调函数通知 同步模式：函数返回即表示命令发送结果 建议运动时以 50Hz 下发命令

3.1.6.17 原地左右翻滚(Turn)

函数名	<code>std::error_code Turn(int direction, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	控制机器狗在原地进行左右翻滚动作
参数	direction: 0 为恢复；1 为左翻滚；2 为右翻滚 timeout_ms: 0 表示异步模式；>0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数；同步模式下不使用
返回值	- `std::errc::success`: 操作成功 - 常见失败错误码： - `std::errc::timed_out`: 操作超时 - `std::errc::not_connected`: 未连接 - `std::errc::operation_canceled`: 操作已取消
备注	仅“原地模式”有效

3.1.6.18 控制头部(ControlHead)

函数名	<code>std::error_code ControlHead(float left_right, float up_down, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	控制机器人“左右探头”与“抬头/低头”动作（仅**原地模式**有效）。
参数	left_right: 取值范围[-1.0, 1.0]，正数:左探头；负数:右探头，单位是 rad/s up_down: 取值范围[-1.0, 1.0]，正数:抬头；负数:低头，单位是 rad/s timeout_ms: 0 表示异步模式；>0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数；同步模式下不使用
返回值	- `std::errc::success`: 操作成功 - 常见失败错误码： - `std::errc::timed_out`: 操作超时 - `std::errc::not_connected`: 未连接 - `std::errc::operation_canceled`: 操作已取消
备注	仅“原地模式”有效

3.1.6.19 设置高低站姿(HighLowStance)

函数名	<code>std::error_code HighLowStance(int stance, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	控制机器人高低站姿，仅“原地模式”有效
参数	stance: 0:恢复; 1:高站姿; 2:低站姿 timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	

3.1.8 数据配置接口

3.1.6.20 配置 IMU(SetImuConfig)

函数名	<code>std::error_code SetImuConfig(int freq, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	配置 IMU 上报频率，默认关闭。
参数	freq: 取值范围[0, 100], 0 表示关闭 timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	

3.1.6.21 配置光强值(SetLuxConfig)

函数名	<code>std::error_code SetLuxConfig(bool on, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	配置光强值上报开关，默认关闭。
参数	on: true:开启; false:关闭 timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	配置后光强值以 1Hz 上报

3.1.6.22 配置运动数据(SetMcConfig)

函数名	<code>std::error_code SetMcConfig(bool on, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	配置运动数据上报开关，默认关闭。
参数	on true:开启; false:关闭 timeout_ms: 0 表示异步模式; >0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数； 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	配置后运动数据以 50Hz 上报。

3.1.6.23 配置速度数据(SetSpeed ReportConfig)

函数名	<code>std::error_code SetSpeedReportConfig(bool on, uint32_t frequency, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	配置速度数据上报开关，默认关闭。配置完成后，将上报速度数据。
参数	on true:开启; false:关闭 frequency: 上报频率，[1hz,50hz] timeout_ms: 0 表示异步模式; >0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数； 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	配置后速度数据以设定的频率上报。

3.1.6.24 获取控制权(TakeControl)

函数名	<code>std::error_code TakeControl(int timeout_ms = 0,</code>
功能概述	获取机器人的控制权。如果当前 APP 在控制，则无法抢占接管。
参数	timeout_ms: 0 表示异步模式; >0:同步模式，最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数； 同步模式下不使用
返回值	- <code>`std::errc::success`</code> : 操作成功 - 常见失败错误码: - <code>`std::errc::timed_out`</code> : 操作超时 - <code>`std::errc::not_connected`</code> : 未连接 - <code>`std::errc::operation_canceled`</code> : 操作已取消
备注	

3.1.6.25 释放控制权(ReleaseControl)

函数名	<code>std::error_code ReleaseControl(int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {})</code>
功能概述	释放机器人的控制权
参数	timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码: - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	v0.0.6 版本 sdk 新增

3.1.6.26 更新摄像头码率

函数名	<code>std::error_code UpdateCameraBitrate(CameraBitrateCmd cmd, int timeout_ms = 0, WriteHandler handler = [] (const std::error_code&, std::size_t) {});</code>
功能概述	更新摄像头码率
参数	cmd: 配置具体摄像头的码率 timeout_ms: 0 表示异步模式; >0:同步模式, 最大等待时间(毫秒) handler: 异步模式下命令发送结果回调函数; 同步模式下不使用
返回值	- 'std::errc::success': 操作成功 - 常见失败错误码: - 'std::errc::timed_out': 操作超时 - 'std::errc::not_connected': 未连接 - 'std::errc::operation_canceled': 操作已取消
备注	v0.0.6 版本 sdk 新增

3.1.9 版本信息

3.1.6.27 获取 sdk 版本号

函数名	<code>const std::string& Version() const</code>
功能概述	获取 SDK 版本号字符串
参数	无
返回值	
备注	

3.1.6.28 获取协议版本号

函数名	<code>const std::string& ProtocolVersion() const</code>
功能概述	获取协议版本号字符串
参数	无
返回值	
备注	

3.1.6.29 获取系统版本号

函数名	const std::string& SystemVersion() const
功能概述	获取系统版本号字符串
参数	无
返回值	
备注	

示例代码

C++

```
#include "robot_sdk/sdk_client.hpp"
using namespace robot_sdk;

int main() {
    SDKClient client([](const std::error_code& ec) {
        if (ec) std::cerr << "SDK Error: " << ec.message() <<
std::endl;
    });

    auto ec = client.Connect("192.168.234.1", "8081", true);
    if (ec) {
        std::cerr << "Connect failed: " << ec.message() <<
std::endl;
        return -1;
    }

    client.StandUp(2000);
    client.Move(0, 0.5, 0);
    client.Disconnect(true);
    return 0;
}
```

注：更多 sdk 数据接口，请以 sdk 文件为准

四、传感设备数据获取

4.1 视频流数据

前后广角相机的视频分别以 Gstreamer 方式推流到以下 URL:

前置摄像头: `rtsp://192.168.234.1:8554/front`

后置摄像头: `rtsp://192.168.234.1:8554/back`

默认会在遥控器侧拉流显示实时画面, 可通过 VLC 或者 ffplay 进行预览和调用, 例如:

Way1: 在 windows 电脑连接四足机器人 wifi, 可通过 VLC 软件进行视频流数据查看:

step1: 打开 vlc, 点击"媒体-->打开网络串流"

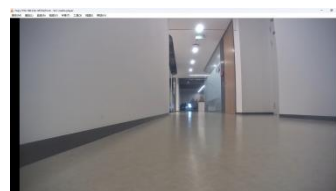


step2: 输入设备推流地址:

`rtsp://192.168.234.1:8554/front`



step3: 预览画面



Way2: ubuntu 电脑连接四足机器人 wifi, 在终端中输入以下指令进行视频流获取:

Bash

```
ffplay rtsp://192.168.234.1:8554/front
ffplay rtsp://192.168.234.1:8554/back
```

注意: 若以低延迟获取视频流数据, 可按如下方法获取:

Bash

```
gst-launch-1.0 rtspsrc location=rtsp://192.168.234.1:8554/front
latency=0 ! rtph264depay ! h264parse ! avdec_h264 !
videoconvert ! videoscale ! video/x-raw,width=1280,height=720 !
autovideosink
```

4.2 激光雷达数据

4.2.1 ROS2 接口定义

topic	msg	字段类型	字段	字段释义	初始 值	数值 范围	发送 周期
前雷达点云数据： /front_lidar 后雷达点云数据： /rear_lidar	sensor_msgs::msg::PointCloud2 (数据服务模型 Qos::best_effort)	std_msgs/Header	header	ROS2 标准 MSG 帧头			10hz
		uint32	height	点云的垂直高度（行数）			
		uint32	width	点云的水平方向宽度（列数）			
		PointField[]	fields	点云中每个通道的字段描述			
		bool	is_big_endian	是否是大端字节序			
		uint32	point_step	每个点所占字节数			
		uint32	row_step	一行点数据占用的字节数			
		uint8[]	data	点云的原始二进制数据，是一维数组			
		bool	is_dense	是否为稠密点云（即没有无效点）			

前激光雷达 IP: 192.168.1.102

后激光雷达 IP: 192.168.2.102

4.2.2 获取激光雷达数据

Bash

```
ssh robot@192.168.168.100
ros2 topic echo /front_lidar --once
ros2 topic echo /rear_lidar --once
```

4.3 超声波雷达数据

4.3.1 ROS2 接口定义

topic	msg	字段类型	字段	字段释义	初始值	数值范围	发送周期
左侧超声波雷达： /uss_driver/uss_left/range 右侧超声波雷达： /uss_driver/uss_right/range	sensor_msgs::Range	std_msgs/Header	header	ROS2 标准 MSG 帧头			10hz
		uint8_t	radiation_type	传感器类型。枚举值：0 超声波 1 红外线	0	0-1	
		float	field_of_view	传感器视场角（弧度），通常为测距锥形区域的角度	0	当前固定为 0	
		float	min_range	传感器可测量最小距离（米）	0	当前固定为 0	
		float	max_range	传感器可测量的最大距离（米）	0	当前固定为 0	
		float	range	当前实际测得距离（米）	0	0-4 米	

4.3.2 获取超声波雷达数据

Bash

```
ros2 topic echo /uss_driver/uss_left/range --once
ros2 topic echo /uss_driver/uss_right/range --once
```

4.5 RTK 数据

4.5.1 RTK 简介

LK1220D 是六分科技自研的一款工规级全频高精度定位定向模组，最多支持 1040 卫星通道,支持双天线接入。模组可内置高精度 NRTK&定向算法，且具备优秀的抗干扰能力，通过基带内置多频窄带抗干扰单元，提升复杂电磁环境下的信号接收和处理性能。

D1 Max 设备内部预留 RTK 拓展接口，且已安装驱动，如需使用 RTK 服务，需外接天线(SMA 接头)并开通 RTK 服务账号，另外需要通过 4G 模组访问外网，方可正常使用 RTK 功能。推荐的 rtk 天线型号：北天 BT560。

定位精度：

水平定位精度: 单点定位 1.5m; 差分定位 2~3cm; RTK 精度: 1cm+1ppm;

高程定位精度: 单点定位 2.5m; 差分定位 4~5cm; RTK 精度: 1.5cm+1ppm;

注 1: 单点定位需加装天线；差分定位需加装天线并开通 RTK 服务账号；

注 2: RTK 精度测量条件：需要高精度天线，空旷区域，距离基站 1km 内；

4.5.2 获取六分 RTK 服务

使用从注册的六分账户和密码登录到官网

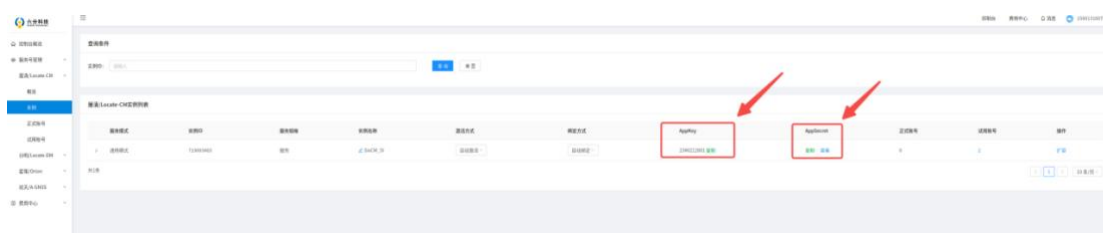
网址: <https://www.sixents.com/login>

- 登录进控制台



● 获取 AK 和 AS

准备好需要使用的 AK 和 AS



4.5.3 NX 侧 RTK 服务配置

● 登录进入 NX

首先需要连接机器狗的 WIFI，密码默认是 12345678

Bash

```
ssh robot@192.168.234.1 #密码是 bot
```

● 打开配置文件

配置文件路径: /ota/sixents_config.ini

Bash

```
sudo vim /ota/sixents_config.ini
```

TOML

配置文件说明

```
[sixents_sdk]
```

```
sdk_ak =
```

```
sdk_as =
```

```
sdk_dev_id =
```

```
sdk_dev_type =
```

■ sdk_ak: 设置您购买服务实例的 AK

- `sdk_as`: 设置您购买服务实例的 AS
- `sdk_dev_id`: 设置您的设备 ID 或 SN（自定义）
- `sdk_dev_type`: 设置您的设备类型（自定义）

注意：`sdk_dev_id` 和 `sdk_dev_type` 是和 账户唯一绑定的，之后更改其他的需要去官网解绑

4.5.4 ROS2 接口定义

- rtk 话题名称: `/rtk_pvh`
- 数据说明

类别	字段	说明
heading	<code>utc_time_s</code>	heading UTC 时间
	<code>sol_status</code>	解的状态 0: 已解出 1: 观测量不足 2: 未收敛 4: 协方差矩阵的迹超过最大值（迹 > 1000 米）
	<code>heading_type</code>	定位类型 0: 不进行解算 16: 单点定位 17: 伪距差分定位 34: 浮点解 50: 整数解
	<code>base_line</code>	基线长度，单位米（0~3000）
	<code>heading_deg</code>	航向角，单位度（0~360）
	<code>pitch_deg</code>	俯仰角，单位度（±90）
	<code>heading_std</code>	航向角标准差，单位度
	<code>pitch_std</code>	俯仰角标准差，单位度

	sys_num	跟踪的卫星数
	soln_svs_num	参与解算的卫星数
bestnav	utc_time_s	bestpos UTC 时间
	p_sol_status	解的状态 0: 已解出 1: 观测量不足 2: 未收敛 4: 协方差矩阵的迹超过最大值 (迹 > 1000 米)
	pos_type	定位类型 0: 不进行解算 16: 单点定位 17: 伪距差分定位 34: 浮点解 50: 整数解 pose_type 是 50 说明是精准定位
	latitude_deg	纬度, 单位度
	longitude_deg	经度, 单位度
	altitude_m	海拔高, 单位米
	undulation	大地水准面差距, 大地水准面与 WGS84 参考椭球面的差值, 单位米
	lat_std	纬度标准差, 单位米
	lon_std	经度标准差, 单位米
	hgt_std	高度标准差, 单位米
	diff_age_s	差分龄期, 单位秒
	sol_age_s	解的龄期, 单位秒

	svs_num	跟踪的卫星数
	soln_svs_num	参与解算的卫星数
	v_sol_status	解的状态
	vel_type	定位类型
	hor_spd	水平方向对地速度，单位米/秒
	trk_gnd	相对于真北的实际对地运动方向，单位度
	ver_spd	垂直方向速度，单位米/秒。正值表示方向向上，负值表示方向向下
	ver_spd_std	垂直方向速度标准差（未填写）
	hor_spd_std	水平方向对地速度标准差（未填写）

4.5.5 获取 RTK 数据

Bash

```
ssh robot@192.168.168.100
ros2 topic echo /rtk_pvh --once
```

4.6 IMU 数据

目前系统中有三个 IMU（2 雷达+1 车规级），通过 topic 发出数据，标准 ROS2 的数据类型 `sensor_msgs::msg::Imu`

前雷达 IMU: `/front_lidar/imu`

后雷达 IMU: `/rear_lidar/imu`

车规级 IMU: `/imu_driver/imu_central`

注：车规级 IMU 涉及硬件改动的问题，若无法获取 imu 数据，请联系售后服务人员

4.7 相机数据

相机数据来源于 RK3588, topic 名称如下：

- 前: `/front_camera/image_compressed`
- 后: `/rear_camera/image_compressed`

五、调试说明

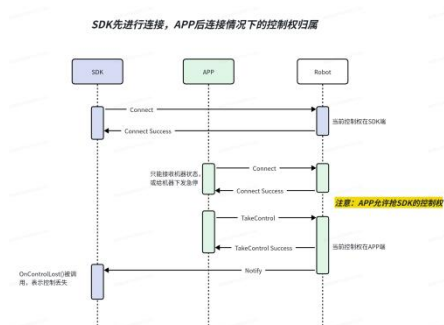
5.1 控制权切换说明

功能说明：控制权切换功能主要用来在进行 sdk 控制阶段，当出现一些紧急情况或需要人工介入处理的场景。

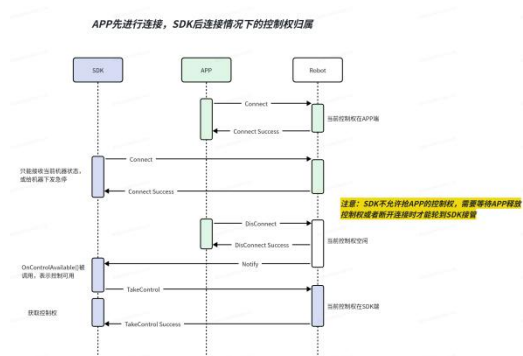
控制原则：

- 在同一时间，不支持遥控器和 sdk 同时进行控制；
- sdk 控制阶段，可以在遥控器端 APP 进行控制权的抢夺，并可在遥控器端触发急停；
- 遥控器控制阶段，sdk 无法获得控制权，但可通过 sdk 下发急停指令并获取设备状态；

SDK 先链接，APP 后链接



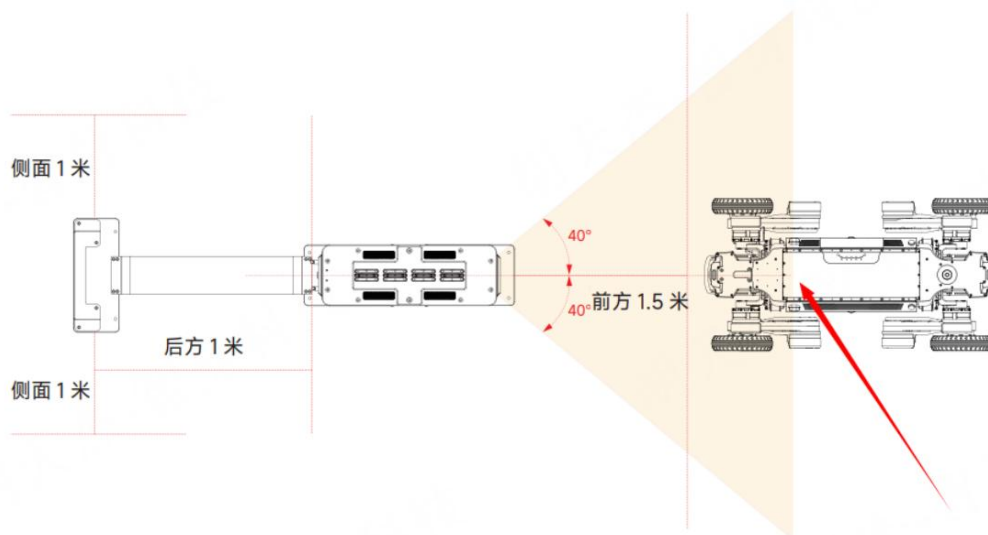
APP 先链接，SDK 后链接



5.2 自主回充说明

版本要求：rk3588 版本为: 0.2.4，Orin NX 固件版本为: 0.4.5

使用要求：需搭配充电桩进行使用，安装说明详见“智元 D1 Max 自主充电产品说明书”，目前支持“无图回充”，即将四足机器人 D1 Max 设备正对充电桩，设备头部距离充电桩 1.5m 左右，确保在相机视野中可完整看到充电桩的二维码。



警示提示

- 充电桩附近不要有其他二维码干扰。
- 安装后请勿遮挡二维码。
- 请勿将二维码处于过曝环境中。

Demo 运行:

Bash

```
# 进入 C++ 演示目录
cd example
```

```
# 创建构建目录
mkdir build && cd build
```

```
#配置并编译项目
cmake ..
make -j6
```

```
#运行 demo 示例
./control 192.168.168.168 8081 192.168.168.100 10010
```

5.3 ROS2 多机通信配置

NX 系统中有统一配置的环境变量：`/opt/runtime/env.bash`，这个变量在 `.bashrc` 中会默认导入

```
rootfs > opt > runtime > env.bash
You, 3周前 | 1 author (You)
1 export PATH=$PATH:/opt/runtime/bin
2 export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/runtime/lib
3
4 source /opt/ros/humble/setup.bash
5 source /usr/share/colcon_cd/function/colcon_cd.sh
6 source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash
7 export ROS_DOMAIN_ID=24
8 export ROS_LOCALHOST_ONLY=0
9 export LC_NUMERIC="en_US.UTF-8"
10 export _colcon_cd_root=/opt/ros/humble/
11 export RMW_IMPLEMENTATION=rmw_zenoh_cpp #rmw_fastdds_cpp
12 export ZENOH_ROUTER_CONFIG_URI=/opt/runtime/config/zenoh_router_config.json5
13 export RCUTILS_COLORIZED_OUTPUT=1
14 export RCUTILS_LOGGING_USE_STDOUT=1
15 export RCUTILS_LOGGING_BUFFERED_STREAM=1
16 export RCUTILS_CONSOLE_OUTPUT_FORMAT="[date] [{severity}] [{name}]: {message}"
17 export ROBOT_LOG_DIR=/home/jszr
18 # export ROBOT_API_HOST=0.0.0.0
```

由上图可见，`ROS_DOMAIN_ID=24`，目前机器狗使用 `ros2` 的 `zenoh` 中间件对外提供 `ros2 topic` 服务，需要用户在自己的笔记本安装相关环境。

● IP 配置

配置用户笔记本 ip 为：192.168.168.xxx（xxx 不能为 100 或者 168 或者 255）

● 用户笔记本环境配置

Ubuntu22.04

Ros2 humble

Zenoh rmw

● 安装 zenoh

Plain Text

```
sudo apt-get install ros-humble-rmw-zenoh-cpp
```

● 修改配置文件

使用编辑器打开下面文件

`/opt/ros/humble/share/rmw_zenoh_cpp/config/DEFAULT_RMW_ZENOH_ROUTER_CONFIG.json5`

找到 `connect/endpoints` 字段，将机器狗 NX 的 IP 写入并保存：

```

rootfs > opt > runtime > config > zenoh_router_config.json5
4  {
29  /// For TCP/UDP links, it's possible to specify the DSCP field of the IP
30  /// E.g. tcp/192.168.0.1:7447#dscp=0x08
31  connect: {
32    /// timeout waiting for all endpoints connected (0: no retry, -1: inf)
33    /// Accepts a single value (e.g. timeout_ms: 0)
34    /// or different values for router, peer and client (e.g. timeout_ms:
35    timeout_ms: { router: -1, peer: -1, client: 0 },
36
37    /// The list of endpoints to connect to.
38    /// Accepts a single list (e.g. endpoints: ["tcp/10.10.10.10:7447", "t
39    /// or different lists for router, peer and client (e.g. endpoints: {
40    ///
41    /// See https://docs.rs/zenoh/latest/zenoh/config/struct.EndPoint.html
42    endpoints: [
43      // "<proto>/<address>"
44      "tcp/192.168.168.100:7447",
45    ],
46
47    /// Global connect configuration,

```

Plain Text

```
"tcp/192.168.168.100:7447"
```

在 pc 端新建终端 1，设置中间件切换的环境变量以及 ROS2 的 Domain ID

Bash

```

source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=24
export RMW_IMPLEMENTATION=rmw_zenoh_cpp
ros2 run rmw_zenoh_cpp rmw_zenohd

```

重置 ROS2 后台守护进程

Bash

```

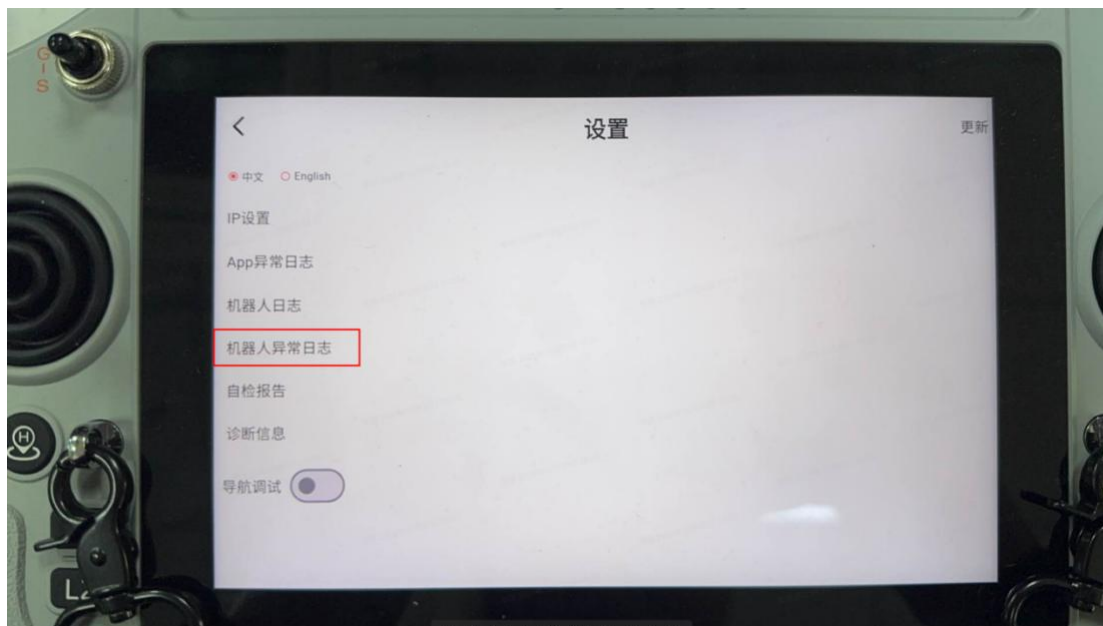
ros2 daemon stop
ros2 daemon start

```

在上述的环境变量生效的前提下即可使用 rviz 可视化，录包，或者启动自己的 ros2 程序与狗中的驱动交互。

5.4 日志查看

- 遥控器报错信息查看



- 设备 log 查看

最近 1 次设备启动后的 log 路径(RK&NX): `/home/robot/robot_launch_log`

最近 6 次设备启动后的 log 路径(RK&NX): `/userdata/log/`

自主回充日志路径(NX): `/tmp/log/alg_data`

Bash

`scp -r robot@192.168.234.1:/userdata/log .` #拉取最近 6 次设备启动后的所有日志

`scp -r robot@192.168.234.1:/home/robot/robot_launch_log .` # 拉取当前设备启动后的日志

```
robot@robot:~$ ls
Desktop Documents Downloads Music Pictures Public robot_launch_log Templates Videos wenxi_dir
robot@robot:~$ ls robot_launch_log
motion_control.stderr      robot_diagnostic_analyzer.stdout  robot_launch.log      robot_monitor.stderr
motion_control.stdout      robot_hal.stderr                  robot_manager.stderr   robot_monitor.stdout
robot_diagnostic_analyzer.stderr  robot_hal.stdout                  robot_manager.stdout   robot_remote.stderr
robot@robot:~$
```

5.5 设备进程工具使用

使用说明：在设备内部通过 `robot-launch` 工具对相关进程进行管理，包括查看进程运行状态、启动和停止进程，查看进程日志信息等。

格式说明：`robot-launch [OPTIONS] [COMMAND]`

```
jszr@orin-nx:~$ robot-launch help
机器人相关程序启动期管理器

Usage: robot-launch [OPTIONS] [COMMAND]

Commands:
  server      以服务器形式启动
  list        显示当前所有的 egg
  egg         显示指定 egg 的快照信息
  stop        停止多个正在运行中的 egg
  start       启动多个停止了的 egg
  restart     重启一个运行中的 egg
  remove      移除多个 egg
  collect     收集|启动一个 egg
  collects    收集|启动多个 egg
  add-service 收集|启动一个 service
  start-service 启动一个 service
  stop-service 关闭一个 service
  restart-service 重启一个 service
  service     显示一个 service 的详细信息
  remove-service 移除一个 service
  example-egg 展示 demo
  example-service 展示 service demo
  stdout-log  out log tail
  stderr-log  error log tail
  auto-complete 生成自动补全脚本
  help       Print this message or the help of the given subcommand(s)

Options:
  -i, --ip <IP>      Target IP address
  -p, --port <PORT>   Target port
  -h, --help          Print help
  -V, --version       Print version
```

eg:

关闭节点：`robot-launch stop <# id>`

打开节点：`robot-launch start <# id>`

查看节点状态：`robot-launch egg <# id>`

查看帮助：`robot-launch help`

...

5.6 连接外部 wifi

Bash

```
ssh robot@192.168.234.1      #登录到 rk 中
wpa_passphrase <WIFI 热点名称> <WIFI 热点密码> > /tmp/wifi.conf
sudo wpa_supplicant -B -i wlan0 -c /tmp/wifi.conf
```

```
robot@robot:/tmp$ wpa_passphrase Agibot-guest Zyjqr0227 > /tmp/wifi.conf
robot@robot:/tmp$ ls
config-err-8MDr8c
hostapd.conf
launch_params_6v3090sa
launch_params_wtv4w32j
mediamtx.log
ssh-XXXXXXQCkKmM
systemd-private-82caab1629f24870a7f15bc38bfb782d-bluetooth.service-0k0QS8
systemd-private-82caab1629f24870a7f15bc38bfb782d-chrony.service-H9T3Kw
systemd-private-82caab1629f24870a7f15bc38bfb782d-systemd-logind.service-H5D6J2
systemd-private-82caab1629f24870a7f15bc38bfb782d-systemd-resolved.service-AnAny0
systemd-private-82caab1629f24870a7f15bc38bfb782d-upower.service-MDEhbU
test
wifi.conf
robot@robot:/tmp$ cat wifi.conf
network={
    ssid="Agibot-guest"
    #psk="Zyjqr0227"
    psk=f6d0acd84e1b5791737742f904a116fdeffa7b2d2cbe067c6840f35a982d9888
}
robot@robot:/tmp$ sudo wpa_supplicant -B -i wlan0 -c /tmp/wifi.conf
Successfully initialized wpa_supplicant
robot@robot:/tmp$ ping www.baidu.com
PING www.a.shifen.com (153.3.238.28) 56(84) bytes of data.
64 bytes from 153.3.238.28 (153.3.238.28): icmp_seq=1 ttl=53 time=9.31 ms
64 bytes from 153.3.238.28 (153.3.238.28): icmp_seq=2 ttl=53 time=10.0 ms
^C
--- www.a.shifen.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 9.313/9.667/10.021/0.354 ms
```

5.7 连接 WiFi 的情况下访问有线网口

连接设备的 AP 热点: XG2WIFI_xxxxx, 添加如下路由可实现在连接 wifi 的情况下能够访问设备的有线网口:

windows 系统:

Bash

```
route -p ADD 192.168.168.0 MASK 255.255.255.0 192.168.234.1
ping 192.168.168.168
ping 192.168.168.100
```

linux 系统:

Bash

```
sudo ip route add 192.168.168.0/24 via 192.168.234.1
ping 192.168.168.168
ping 192.168.168.100
```

